

**PENGEMBANGAN SIEM BERBASIS WAZUH DENGAN
INTEGRASI MACHINE LEARNING UNTUK DETEKSI
ANOMALI KEAMANAN PADA WEB SERVER**

TUGAS AKHIR

Oleh :

MUHAMMAD RAZIE

2201082009

Acc Sidang
25/02/26



Acc Sidang TA
25/2/26

**PROGRAM STUDI D3 TEKNIK KOMPUTER
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI PADANG**

2026

**PENGEMBANGAN SIEM BERBASIS WAZUH DENGAN
INTEGRASI MACHINE LEARNING UNTUK DETEKSI
ANOMALI KEAMANAN PADA WEB SERVER**

TUGAS AKHIR

Diajukan sebagai salah satu syarat untuk mendapatkan gelar Ahli Madya pada Jurusan
Teknologi Informasi Program Studi D3 Teknik Komputer

Oleh :

MUHAMMAD RAZIF

2201082039



**PROGRAM STUDI D3 TEKNIK KOMPUTER
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI PADANG**

2026

**PENGEMBANGAN SIEM BERBASIS WAZUH DENGAN
INTEGRASI MACHINE LEARNING UNTUK DETEKSI
ANOMALI KEAMANAN PADA WEB SERVER**

TUGAS AKHIR

Diajukan sebagai salah satu syarat untuk mendapatkan gelar Ahli Madya pada
Jurusan Teknologi Informasi Program Studi D3 Teknik Komputer :

Oleh :

MUHAMMAD RAZIF

2201082039

Tugas Akhir ini telah diperiksa dan disetujui oleh :

Pembimbing I

Pembimbing II

Deddy Prayama, S.kom., M.ISD
NIP. 19810415 200604 1 002

Fanni Sukma, S.ST., M.T
NIP. 19900606 201903 2 026

Disahkan Oleh :

Ketua Jurusan
Teknologi Informasi

Ketua Program Studi
Teknik Komputer

Dr.Ir. Yuhefizar, S.Kom., M.Kom
NIP. 19760113 200604 1002

Andre Febian Kasmar, S.T., M.T
NIP. 19880220 201903 1 009

**PENGEMBANGAN SIEM BERBASIS WAZUH DENGAN
INTEGRASI MACHINE LEARNING UNTUK DETEKSI
ANOMALI KEAMANAN PADA WEB SERVER**

Oleh :

MUHAMMAD RAZIF

2201082039

Tugas Akhir ini telah diuji dan dipertahankan didepan tim
penguji sidang Tugas Akhir Diploma III Politeknik Negeri Padang
Pada hari Jumat Tanggal 27 Februari 2026

Tim Penguji,

Ketua

Sekretaris

Deddy Prayama, S.kom., M.ISD

NIP. 19810415 200604 1 002

Ismael, S.Kom, M.Kom

NIP. 19850319 202321 1 009

Anggota

Anggota

Ideva Gaputra, S.Kom., M.Kom

NIP. 19880912 202203 1 006

Taruma Leo Wijaya, S.Kom., M.M.S.I

NIP. 19850319 202321 1 009

ABSTRAK

Perkembangan layanan berbasis web meningkatkan risiko serangan siber pada web server sehingga diperlukan sistem monitoring keamanan yang lebih adaptif. Penelitian ini mengembangkan sistem Security Information and Event Management (SIEM) berbasis Wazuh yang diintegrasikan dengan algoritma Machine Learning Random Forest untuk membantu klasifikasi anomali pada web server Ubuntu 22.04. Sistem dibangun pada lingkungan Amazon Web Services (AWS) dengan komponen Wazuh Manager, Wazuh Agent, Elasticsearch, dan Machine Learning Engine. Log keamanan dikumpulkan dan diproses secara terpusat, kemudian digunakan sebagai dataset untuk pelatihan dan pengujian model. Tahapan penelitian meliputi pengambilan data dari Elasticsearch, preprocessing dan feature engineering, pelatihan model, evaluasi menggunakan confusion matrix, serta implementasi deteksi realtime dengan notifikasi Telegram. Hasil pengujian pada dataset simulasi menunjukkan model memiliki performa klasifikasi yang sangat baik dalam membedakan aktivitas normal dan anomali. Integrasi machine learning memberikan lapisan analisis tambahan di atas sistem rule-based Wazuh sehingga membantu memvalidasi alert dan mengurangi beban analisis manual administrator.

Kata Kunci: SIEM, Wazuh, Machine Learning, Random Forest, Deteksi Anomali.

ABSTRACT

The rapid growth of web-based services increases the risk of cyberattacks on web servers, requiring a more adaptive and integrated security monitoring system. This research develops a Security Information and Event Management (SIEM) system based on Wazuh integrated with a Random Forest machine learning algorithm to assist in classifying normal and anomalous activities on an Ubuntu 22.04 web server. The system is deployed in a cloud environment using Amazon Web Services (AWS) and consists of Wazuh Manager, Wazuh Agent, Elasticsearch, and a Machine Learning Engine. Security logs are centrally collected and stored in Elasticsearch, then used as a dataset for preprocessing, feature engineering, model training, and evaluation using a confusion matrix. The trained model is further implemented in a real-time detection mechanism integrated with Telegram notifications. Testing on a simulated dataset shows that the model achieves strong classification performance in distinguishing normal and anomalous activities. The integration of machine learning provides an additional analytical layer on top of the rule-based system, helping validate alerts and reduce manual analysis workload for administrators.

Keywords: *SIEM, Wazuh, Machine Learning, Random Forest, Anomaly Detection.*

DAFTAR ISI

HALAMAN PENGESAHAN.....	i
HALAMAN PENGUJI.....	ii
ABSTRAK.....	iii
ABSTRACT.....	iv
DAFTAR ISI.....	v
DAFTAR TABEL.....	vii
DAFTAR GAMBAR.....	viii
DAFTAR LAMPIRAN.....	x
KATA PENGANTAR.....	xi
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan	3
1.4 Manfaat	3
1.5 Batasan Masalah.....	4
1.6 Metodologi	5
BAB II LANDASAN TEORI.....	9
2.1 Penelitian Terdahulu	9
2.2 Teori Tentang Sistem.....	10
2.2.1 SIEM (<i>Security Information and Event Management</i>)	10
2.2.2 Web Server.....	10
2.2.3 Apache Web Server	11
2.2.4 Wazuh	12
2.2.5 AWS.....	13
2.2.6 Elasticsearch	14
2.2.7 Kibana.....	14
2.2.8 Machine Learning	15
2.2.9 Python	19
2.2.10 Telegram Bot	20
2.2.11 Nmap.....	20
2.2.12 Metasploit	21
BAB III ANALISIS DAN PERANCANGAN	20
3.1 Analisis Sistem yang Sedang Berjalan.....	20

3.1.1	Analisis Kebutuhan Perangkat Keras.....	20
3.1.2	Analisis Kebutuhan Perangkat Lunak.....	22
3.2	Rancangan Sistem.....	24
3.2.1	Rancangan Topologi	24
3.2.2	Rancangan Flowchart.....	25
BAB IV IMPLEMENTASI DAN PENGUNJIAN		28
4.1	Implementasi.....	28
4.2	Arsitektur Infrastruktur AWS	28
4.2.1	Wazuh Manager dan Wazuh Dashboard.....	28
4.2.2	Web Server dan Wazuh Agent.....	29
4.2.3	Elasticsearch	29
4.2.4	Machine Learning	29
4.3	Pembuatan dan Konfigurasi AWS	29
4.3.1	Instance Wazuh Manager.....	29
4.3.2	Instance Wazuh Agent (WEB Server)	33
4.3.3	Instance Elasticsearch	36
4.3.4	Instance Machine Learning.....	40
4.4	Implementasi Wazuh.....	43
4.4.1	Instalasi Wazuh Manager dan Wazuh Dashboard.....	43
4.4.2	Instalasi dan Konfigurasi Wazuh Agent	46
4.5	Integrasi Penyimpanan Log ke Elasticsearch.....	47
4.5.1	Arsitektur Pengiriman Log.....	48
4.5.2	Struktur Data Log dalam Elasticsearch.....	49
4.5.3	Keuntungan Integrasi Elasticsearch dalam Sistem.....	49
4.6	Implementasi Kibana sebagai Dashboard Monitoring	50
4.7	Implementasi dan Integrasi Machine Learning.....	52
4.8	Perbandingan Sistem Wazuh Tanpa Machine Learning dan Wazuh dengan Machine Learning	63
4.8.1	Perbandingan Arsitektur Sistem.....	63
4.8.2	Perbandingan Mekanisme Deteksi.....	63
4.8.3	Perbandingan Kuantitatif	65
4.8.4	Kesimpulan Perbandingan	68
BAB V PENUTUP		70
5.1	Kesimpulan	70
5.2	Saran	71
DAFTAR PUSTAKA		70
LAMPIRAN.....		73

DAFTAR TABEL

Tabel 3. 1 Spesifikasi Instance Wazuh Manager + Dashboard.....	20
Tabel 3. 2 Spesifikasi Instance Elasticsearch.....	21
Tabel 3. 3 Spesifikasi Instance Wazuh Agent (Web Server)	21
Tabel 3. 4 Spesifikasi Instance Machine Learning Engine	22
Tabel 3.5 Analisis kebutuhan perangkat lunak	22
Tabel 3.6 Layanan AWS yang digunakan	23
Tabel 4. 1 Daftar Fitur Machine Learning	54

DAFTAR GAMBAR

Gambar 1. 1 Diagram Topologi	6
Gambar 1. 2 Desain Alur Data.....	6
Gambar 2. 1 Wazuh	13
Gambar 2. 2 Amazon Web Service.....	14
Gambar 2. 3 Elasticsearch.....	14
Gambar 2. 4 Kibana	15
Gambar 2. 5 Algoritma Random Forest.....	16
Gambar 2. 6 Python	20
Gambar 2. 7 Telegram	20
Gambar 2. 8 NMAP	21
Gambar 2. 9 Metasploit.....	21
Gambar 3. 1 Rancangan Topologi	24
Gambar 3. 2 Rancangan Flowchart.....	26
Gambar 4. 1 Launch Instance pada AWS EC2	29
Gambar 4. 2 Konfigurasi Name and Tags Instance	30
Gambar 4. 3 Pemilihan Amazon Machine Image (AMI).....	30
Gambar 4. 4 Pemilihan Tipe Instance	31
Gambar 4. 5 Pembuatan Key Pair.....	31
Gambar 4. 6 Konfigurasi Storage (EBS)	32
Gambar 4. 7 Konfigurasi Elastic IP pada AWS.....	32
Gambar 4. 8 Konfigurasi Security Group dan Port.....	32
Gambar 4. 9 Launch Instance pada AWS EC2	33
Gambar 4. 10 Konfigurasi Name and Tags Wazuh Agent (Web Server)	33
Gambar 4. 11 Pemilihan AMI Wazuh Agent (Web Server)	34
Gambar 4. 12 Pemilihan Tipe Instance Wazuh Agent (Web Server)	34
Gambar 4. 13 Konfigurasi Key Pair Wazuh Agent (Web Server).....	35
Gambar 4. 14 Konfigurasi Storage Wazuh Agent (Web Server).....	35
Gambar 4. 15 Konfigurasi Elastic IP Wazuh Agent (Web Server).....	36
Gambar 4. 16 Konfigurasi Security Group Wazuh Agent (Web Server).....	36
Gambar 4. 17 Launch Instance	36
Gambar 4. 18 Konfigurasi Name and Tags Elasticsearch.....	37
Gambar 4. 19 Pemilihan AMI Elasticsearch.....	37
Gambar 4. 20 Pemilihan Tipe Instance Elasticsearch	38
Gambar 4. 21 Konfigurasi Key Pair Elasticsearch.....	38
Gambar 4. 22 Konfigurasi Storage Elasticsearch	39
Gambar 4. 23 Konfigurasi Elastic IP Elasticsearch	39
Gambar 4. 24 Konfigurasi Security Group Elasticsearch	39
Gambar 4. 25 Launch Instance Machine Learning	40
Gambar 4. 26 Konfigurasi Name and Tags Machine Learning	40
Gambar 4. 27 Pemilihan AMI Machine Learning	41
Gambar 4. 28 Pemilihan Tipe Instance Machine Learning.....	41
Gambar 4. 29 Konfigurasi Key Pair Machine Learning	42
Gambar 4. 30 Konfigurasi Storage Machine Learning	42
Gambar 4. 31 Konfigurasi Elastic IP Machine Learning.....	42
Gambar 4. 32 Konfigurasi Security Group Machine Learning.....	43

Gambar 4. 33 Koneksi SSH Wazuh Manager.....	43
Gambar 4. 34 Update dan Upgrade Wazuh Manager	44
Gambar 4. 35 Instalasi Wazuh Manager.....	44
Gambar 4. 36 Halaman Login Wazuh Dashboard	45
Gambar 4. 37 User dan Password Wazuh Manager.....	45
Gambar 4. 38 Koneksi SSH Menggunakan Wazuh Agent	46
Gambar 4. 39 Update dan Upgrade Wazuh Agent.....	46
Gambar 4. 40 Instalasi Wazuh Agent	47
Gambar 4. 41 Status Wazuh Agent	47
Gambar 4. 42 Alert Dalam Format JSON.....	48
Gambar 4. 43 Konfigurasi Kibana	50
Gambar 4. 44 Create Data View Index Pattern.....	51
Gambar 4. 45 Discover wazuh-alert-*	51
Gambar 4. 46 Script konfigurasi.py	53
Gambar 4. 47 Script Pengambilan Data.....	53
Gambar 4. 48 Preprocessing	54
Gambar 4. 49 Pelatihan Model Random Forest.....	55
Gambar 4. 50 Script Pelatihan Model.....	56
Gambar 4. 51 Evaluasi Random Forest	58
Gambar 4. 52 Deteksi Realtime	60
Gambar 4. 53 Notifikasi Telegram	62
Gambar 4. 54 Confusion Matrix	66

DAFTAR LAMPIRAN

Lampiran 1 Script Pengambilan Dataset.....	73
Lampiran 2 Script Preprocessing dan Feature Engineering	75
Lampiran 3 Script Pelatihan Model	77
Lampiran 4 Script Evaluasi Model	78
Lampiran 5 Script Deteksi Realtime dan Notifikasi Telegram.....	79
Lampiran 6 Alert JSON Wazuh Tanpa Machine Learning.....	82
Lampiran 7 Alert JSON Wazuh dengan Machine Learning	85

KATA PENGANTAR

Puji dan syukur penulis panjatkan kehadirat Allah SWT atas segala rahmat, karunia, dan hidayah-Nya, sehingga penulis dapat menyelesaikan Tugas Akhir ini dengan baik. Tugas Akhir ini disusun sebagai salah satu syarat untuk memperoleh gelar Ahli Madya pada Program Studi Teknik Komputer, Jurusan Teknologi Informasi, Politeknik Negeri Padang.

Penyusunan Tugas Akhir ini merupakan proses pembelajaran yang memberikan pengalaman berharga bagi penulis, baik dari segi akademik maupun pengembangan kemampuan teknis dan analitis. Dalam proses perancangan sistem, implementasi, hingga penyusunan laporan, penulis menyadari sepenuhnya bahwa keberhasilan ini tidak terlepas dari dukungan, arahan, dan bimbingan berbagai pihak. Oleh karena itu, dengan penuh rasa hormat penulis menyampaikan terima kasih kepada:

1. Allah SWT atas segala rahmat, kesehatan, kemudahan, serta kekuatan yang diberikan sehingga penulis mampu menyelesaikan Tugas Akhir ini.
2. Kedua orang tua dan keluarga tercinta yang senantiasa memberikan doa, dukungan, serta motivasi yang menjadi kekuatan utama bagi penulis dalam menyelesaikan pendidikan ini.
3. Bapak Dr. Ir. Yuhefizar, S.Kom., M.Kom., selaku Ketua Jurusan Teknologi Informasi.
4. Bapak Andre Febrian Kasmar, S.T., M.T., selaku Ketua Program Studi Teknik Komputer Politeknik Negeri Padang.
5. Bapak Deddy Prayama, S.Kom., M.ISD., selaku Dosen Pembimbing I yang telah memberikan arahan, bimbingan, masukan teknis, serta pandangan akademik dalam proses perancangan dan pengembangan sistem pada Tugas Akhir ini.
6. Bapak Fanni Sukma, S.ST., M.T., selaku Dosen Pembimbing II yang telah memberikan saran, koreksi, serta evaluasi konstruktif demi penyempurnaan laporan ini.

7. Seluruh dosen dan staf di lingkungan Politeknik Negeri Padang yang telah memberikan ilmu, wawasan, serta pengalaman selama masa perkuliahan.
8. Rekan-rekan mahasiswa yang telah memberikan dukungan dan semangat selama proses penyusunan Tugas Akhir ini.

Penulis menyadari bahwa laporan ini masih memiliki keterbatasan, baik dari sisi teknis maupun penyajian. Oleh karena itu, kritik dan saran yang bersifat membangun sangat diharapkan sebagai bahan evaluasi untuk pengembangan yang lebih baik di masa mendatang. Besar harapan penulis agar Tugas Akhir ini dapat memberikan kontribusi positif dalam pengembangan sistem monitoring dan keamanan jaringan, serta menjadi referensi bagi penelitian selanjutnya.

Padang, 26 Februari 2026

Muhammad Razif
NIM. 2201082039

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah memberikan kemudahan serta efisiensi di berbagai bidang, seperti pemerintahan, pendidikan, bisnis, dan layanan publik. Namun, kemajuan tersebut juga diiringi dengan meningkatnya ancaman keamanan siber yang semakin kompleks dan beragam. Salah satu sasaran utama serangan siber adalah web server, karena berfungsi sebagai gerbang utama layanan berbasis internet. Jika tidak terlindungi dengan baik, web server dapat menjadi titik masuk bagi berbagai serangan, seperti brute-force login, SQL injection, cross-site scripting (XSS), port scanning, hingga unggahan file berbahaya.

Berdasarkan laporan BSSN, tercatat jutaan aktivitas anomali lalu lintas jaringan setiap tahunnya di Indonesia, yang sebagian besar berasal dari aktivitas mencurigakan pada layanan HTTP dan SSH [1]. Dalam konteks keamanan sistem, anomali merupakan aktivitas atau pola perilaku yang menyimpang dari kondisi normal sistem. Contohnya adalah percobaan login berulang dalam waktu singkat, lonjakan trafik yang tidak biasa, atau perubahan file sistem tanpa otorisasi.

Untuk mengatasi permasalahan tersebut, banyak organisasi menerapkan sistem Security Information and Event Management (SIEM). SIEM memungkinkan pengumpulan, korelasi, serta analisis data log secara terpusat dari berbagai perangkat dan aplikasi. Salah satu platform SIEM open-source yang banyak digunakan adalah Wazuh [2]. Wazuh menyediakan fitur deteksi ancaman berbasis aturan (rule-based detection), pemantauan integritas file, serta visualisasi alert melalui dashboard.

Meskipun efektif untuk mendeteksi serangan yang sudah memiliki aturan, sistem berbasis rule memiliki keterbatasan dalam mengenali pola serangan baru atau variasi aktivitas yang belum terdokumentasi. Pendekatan ini kurang adaptif terhadap pola anomali yang kompleks atau tidak terduga [3].

Beberapa penelitian menunjukkan bahwa integrasi machine learning ke dalam sistem SIEM dapat meningkatkan kemampuan deteksi ancaman dan membantu mengurangi false positive dibandingkan pendekatan berbasis aturan saja [4]. Machine learning memungkinkan sistem mempelajari pola historis dari data log dan mengklasifikasikan aktivitas berdasarkan karakteristik data tersebut.

Salah satu algoritma yang cukup stabil untuk klasifikasi data keamanan adalah Random Forest. Algoritma ini mampu menggabungkan beberapa decision tree untuk meningkatkan akurasi serta mengurangi risiko overfitting [5].

Berdasarkan permasalahan dan penelitian terdahulu tersebut, pada tugas akhir ini dikembangkan sistem SIEM berbasis Wazuh yang diintegrasikan dengan algoritma Random Forest sebagai lapisan analisis tambahan untuk membantu klasifikasi aktivitas log menjadi normal dan anomali pada web server berbasis Ubuntu 22.04. Sistem ini juga dilengkapi dengan mekanisme notifikasi otomatis kepada administrator untuk mempercepat respons terhadap potensi ancaman keamanan.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut rumusan masalah dalam tugas akhir ini adalah sebagai berikut:

1. Bagaimana merancang dan mengimplementasikan sistem SIEM berbasis Wazuh untuk melakukan monitoring log keamanan pada web server?
2. Bagaimana proses pengambilan dan pengolahan data log agar dapat digunakan sebagai dataset machine learning?

3. Bagaimana penerapan algoritma Random Forest untuk mengklasifikasikan aktivitas log menjadi normal dan anomali?
4. Bagaimana integrasi hasil klasifikasi machine learning dengan sistem notifikasi otomatis?
5. Bagaimana perbandingan performa sistem Wazuh sebelum dan sesudah integrasi machine learning?

1.3 Tujuan

Dalam melakukan tugas akhir ini terdapat beberapa tujuan yang hendak di capai yaitu:

1. Menerapkan sistem SIEM berbasis Wazuh untuk melakukan pemantauan pada web server berbasis Linux (Ubuntu 22.04).
2. Mengolah data log keamanan menjadi dataset yang dapat digunakan dalam pelatihan machine learning.
3. Menerapkan algoritma Random Forest untuk membantu klasifikasi aktivitas log menjadi normal dan anomali.
4. Mengintegrasikan hasil klasifikasi machine learning dengan sistem notifikasi Telegram.
5. Menganalisis perbandingan performa sistem Wazuh tanpa dan dengan integrasi machine learning.

1.4 Manfaat

Adapun manfaat-manfaat yang dapat diperoleh yaitu:

1. Memberikan solusi deteksi ancaman yang lebih adaptif dan cerdas.
2. Membantu administrator sistem dalam mendeteksi dan merespons potensi ancaman lebih cepat.
3. Menjadi referensi pengembangan sistem SIEM berbasis machine learning.
4. Meningkatkan pemahaman mahasiswa dalam bidang keamanan jaringan dan analisis data log.

5. Menambah literatur ilmiah mengenai penerapan machine learning dalam dunia keamanan jaringan.
6. Menjadi dasar pengembangan penelitian lanjutan dalam bidang deteksi anomali dan keamanan siber.

1.5 Batasan Masalah

Untuk memperjelas masalah yang akan dibahas dan agar tidak menjadi pembahasan yang meluas atau menyimpang maka perlu batasan masalah yaitu sebagai berikut:

1. Web server yang digunakan berbasis Ubuntu Server 22.04 LTS dengan layanan Apache2 (Apache HTTP Server versi default Ubuntu 22.04).
2. Log yang dianalisis terbatas pada log yang dikumpulkan oleh Wazuh Agent, meliputi:
 - a. Aktivitas login SSH (auth.log).
 - b. Perubahan file sistem (syscheck).
 - c. Log sistem dan layanan web (syslog dan apache2 access.log).
3. Simulasi pengujian mencakup port scanning menggunakan Nmap dan perubahan file sistem. Dataset publik seperti UNSW-NB15 atau CIC-IDS2017 bisa digunakan sebagai referensi tambahan dan bukan sebagai sumber utama pelatihan model.
4. Model machine learning yang digunakan dalam penelitian ini dibatasi pada algoritma Random Forest dengan pendekatan supervised learning.
5. Integrasi machine learning dilakukan menggunakan script Python sederhana pada instance Machine Learning tanpa membangun aplikasi web tambahan, dan monitoring visual dilakukan melalui Kibana yang terhubung dengan Elasticsearch.
6. Evaluasi sistem difokuskan pada perbandingan performa deteksi sebelum dan sesudah integrasi machine learning berdasarkan metrik Accuracy, Precision, Recall, dan F1-score, tanpa membahas optimasi lanjutan seperti tuning hyperparameter atau deep learning.

1.6 Metodologi

1. Studi Literatur

Pada tahap ini dilakukan pengumpulan referensi dan literatur dari buku, jurnal, dan dokumentasi resmi yang berkaitan dengan SIEM, Wazuh, serta penerapan machine learning untuk deteksi anomali. Studi ini menjadi dasar dalam menentukan arsitektur sistem dan algoritma yang digunakan.

2. Perancangan Sistem

Perancangan sistem mencakup arsitektur teknis dan alur integrasi antar komponen:

a. Arsitektur Sistem

Infrastruktur cloud (AWS EC2 Ubuntu 22.04), instalasi Wazuh Manager, Agent, Dashboard, dan Elasticsearch.

b. Diagram Topologi

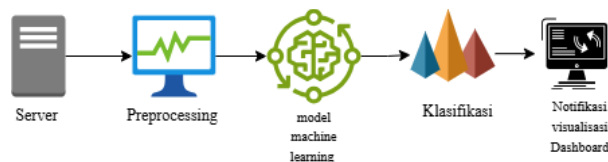
Pada gambar 1.1 menjelaskan alur komunikasi antara komponen yang akan dibangun. Terdapat web server yang menjadi target pemantauan akan mengirimkan catatan semua aktivitas di server ke Wazuh Agent. Selanjutnya, catatan tersebut diteruskan ke Wazuh Manager untuk dianalisis untuk membedakan mana aktivitas normal dan mana berpotensi ancaman dan disimpan pada Elasticsearch. Pada Machine Learning Engine catatan aktivitas (log aktivitas) diproses untuk mendeteksi adanya anomali. Jika terdeteksi aktivitas mencurigakan, sistem akan mengirimkan notifikasi melalui Telegram ke administrator atau pengguna.



Gambar 1. 1 Diagram Topologi

c. Desain Alur Data

Pada gambar 1.2 menjelaskan bagaimana data log atau catatan aktivitas diolah agar bisa menghasilkan informasi yang berguna untuk deteksi serangan dan pengambilan keputusan keamanan. Yang dimana dimulai saat log dikumpulkan di web server kemudian diproses (*preprocessing*), lalu dianalisis oleh model machine learning untuk menentukan aktivitas log tersebut apakah normal atau anomali dan hasilnya ditampilkan di dashboard Wazuh serta notifikasi ke administrator atau pengguna.



Gambar 1. 2 Desain Alur Data

3. Implementasi Sistem

Melakukan pembangunan sistem berdasarkan rancangan yang telah dibuat, meliputi:

- Instalasi dan konfigurasi Wazuh Manager, Wazuh Agent.
- Sinkronisasi log dari web server (auth.log, syslog, apache2.log).
- Simulasi serangan seperti port scanning (Nmap), file modification, dan eksploitasi ringan dengan Metasploit Framework.
- Pengumpulan log hasil simulasi untuk digunakan sebagai dataset machine learning.

4. Pengujian Sistem

Melakukan pengujian terhadap dua skenario utama:

- a. Aktivitas Normal: login biasa, akses file rutin, restart service.
- b. Aktivitas Anomali: port scanning, perubahan file sistem, dan eksploitasi ringan.

Pengujian dilakukan untuk memastikan sistem dapat mengenali perbedaan antara aktivitas normal dan anomali, baik dengan rule-based Wazuh maupun integrasi machine learning.

5. Analisis Hasil

Tahap analisis hasil dilakukan untuk mengevaluasi kinerja sistem deteksi keamanan yang telah dibangun, baik sistem Wazuh berbasis aturan (rule-based) maupun sistem yang telah diintegrasikan dengan machine learning Random Forest. Evaluasi dilakukan menggunakan pendekatan kuantitatif berdasarkan confusion matrix, yang terdiri dari: True Positive Rate (TPR).

- a. True Positive Rate (TPR): jumlah aktivitas anomali yang berhasil terdeteksi dengan benar.
- b. True Negative Rate (TNR): jumlah aktivitas normal yang berhasil diklasifikasikan dengan benar.
- c. False Positive Rate (FPR): aktivitas normal yang salah terdeteksi sebagai anomali.
- d. False Negative Rate (FNR): aktivitas anomali yang tidak berhasil terdeteksi.

Dalam sistem keamanan, nilai False Negative Rate (FNR) memiliki dampak yang lebih berbahaya dibandingkan False Positive, karena aktivitas serangan yang tidak terdeteksi dapat menyebabkan kerusakan sistem atau kebocoran data. Oleh karena itu, analisis tidak hanya berfokus pada akurasi keseluruhan, tetapi juga memperhatikan kemampuan sistem dalam mendeteksi seluruh aktivitas anomali.

Berdasarkan nilai confusion matrix, dihitung beberapa metrik evaluasi, yaitu:

- a. Accuracy, untuk mengetahui tingkat ketepatan sistem secara keseluruhan.
- b. Precision, untuk mengukur seberapa banyak deteksi anomali yang benar-benar merupakan anomali.
- c. Recall, untuk mengukur kemampuan sistem dalam mendeteksi seluruh aktivitas anomali yang ada.
- d. F1-score, sebagai nilai keseimbangan antara precision dan recall.

Recall menjadi metrik yang sangat penting dalam penelitian ini karena berkaitan langsung dengan kemampuan sistem dalam mendeteksi serangan. Sistem dengan akurasi tinggi tetapi recall rendah tetap berisiko karena masih memungkinkan adanya serangan yang tidak terdeteksi.

Selain itu, dilakukan perbandingan jumlah alert dan tingkat kesalahan deteksi antara sistem tanpa machine learning dan sistem dengan machine learning untuk melihat apakah integrasi model Random Forest mampu membantu mengurangi false positive serta memberikan klasifikasi tambahan terhadap alert yang dihasilkan Wazuh. Hasil evaluasi kemudian dianalisis untuk menentukan efektivitas sistem yang dikembangkan dalam meningkatkan kualitas deteksi keamanan pada web server.

BAB II

LANDASAN TEORI

2.1 Penelitian Terdahulu

Dalam penyusunan tugas akhir ini, penulis menggunakan beberapa penelitian sebagai studi literatur dan memiliki keterkaitan terhadap penelitian yang akan dilakukan dimana dapat dijadikan sebagai pembandingan, bahan acuan dan pembelajaran sehingga dapat memudahkan dalam menyelesaikan penelitian. Berikut ini penelitian terdahulu yang berhubungan dengan tugas akhir ini antara lain:

Tabel 2. 1 Ringkasan Penelitian Terdahulu

No	Referensi	Metode	Hasil Penelitian	Perbedaan dengan Penelitian
1	Landauer et al. [3]	Deep Learning (Autoencoder, RNN, CNN) untuk deteksi anomali log	Deep learning meningkatkan akurasi deteksi anomali pada data log skala besar	Penelitian ini menggunakan Random Forest, bukan deep learning
2	Al-Harthy [4]	Integrasi Wazuh dengan supervised & unsupervised learning	False positive turun hingga 78% dibanding rule-based	Penelitian ini hanya menggunakan Random Forest sebagai model supervised
3	Muhammad et al. [6]	Integrasi SIEM + IDS + ML (SVM)	Sistem mampu mendeteksi serangan DoS secara real-time	Penelitian ini fokus pada klasifikasi log Wazuh tanpa IDS tambahan
4	Tendikov	Feature	Akurasi	Penelitian ini

	et al. [7]	selection + AdaBoost, Random Forest	mencapai 97% pada dataset CICIDS2017	menggunakan dataset log langsung dari Wazuh
5	Chamkar et al. [8]	Hybrid Random Forest + DBSCAN	Akurasi 97,2% dan mampu memproses 500 event/detik	Penelitian ini tidak menggunakan metode hybrid

2.2 Teori Tentang Sistem

2.2.1 SIEM (*Security Information and Event Management*)

Beberapa tools monitoring yang berfungsi untuk memberikan peringatan (*Alert*) terkait keamanan pada Agent yang ada pada jaringan cloud adalah *Security Information and Event Management* (SIEM). Teknologi SIEM memiliki cakupan pengumpulan data yang besar serta mampu mengkorelasikan dan menganalisis event dari berbagai sumber dan menentukan apakah kejadian tersebut merupakan suatu serangan atau tidak secara realtime [9].

2.2.2 Web Server

Web server merupakan perangkat lunak yang berfungsi untuk menerima permintaan (request) dari pengguna melalui protokol HTTP atau HTTPS dan mengirimkan halaman web sebagai respon. Web server menjadi komponen utama dalam layanan internet karena berperan sebagai penghubung antara pengguna dengan aplikasi atau layanan berbasis web. Tanpa web server, pengguna tidak dapat mengakses website atau aplikasi melalui jaringan internet [10].

Dalam penelitian ini digunakan Apache HTTP Server pada Ubuntu Server 22.04 sebagai objek monitoring keamanan. Web server sering menjadi target utama serangan siber karena

menyimpan data penting dan menyediakan layanan publik yang dapat diakses melalui internet. Beberapa jenis serangan yang umum terjadi pada web server antara lain brute-force login, SQL Injection, Cross-Site Scripting (XSS), port scanning, dan file upload berbahaya [11].

Untuk meningkatkan keamanan web server, diperlukan sistem monitoring berbasis Security Information and Event Management (SIEM) seperti Wazuh yang mampu mengumpulkan log aktivitas, menganalisis event secara real-time, serta mendeteksi aktivitas mencurigakan. Integrasi machine learning dalam sistem SIEM memungkinkan deteksi pola anomali yang tidak terdeteksi oleh rule-based system saja.

Dengan demikian, web server pada penelitian ini berperan sebagai sumber data log utama yang dianalisis oleh Wazuh dan machine learning untuk mendeteksi anomali keamanan.

2.2.3 Apache Web Server

Apache HTTP Server merupakan salah satu web server open-source yang paling banyak digunakan di dunia karena stabil, fleksibel, dan memiliki dukungan komunitas yang luas. Apache dikembangkan oleh Apache Software Foundation dan mendukung berbagai sistem operasi seperti Linux, Windows, dan macOS [12].

Apache Web Server mampu melayani berbagai jenis layanan web seperti hosting website statis, aplikasi web dinamis, dan layanan API. Apache juga menyediakan fitur keamanan seperti authentication, authorization, SSL/TLS encryption, serta modul keamanan tambahan yang dapat meningkatkan perlindungan terhadap serangan web [13].

Dalam penelitian ini, Apache digunakan sebagai web server pada instance Wazuh Agent. Log yang dihasilkan Apache seperti

access.log dan error.log digunakan sebagai sumber data untuk monitoring keamanan. Log tersebut berisi informasi alamat IP pengguna, waktu akses, metode HTTP, serta status respon server. Data ini sangat penting untuk mendeteksi aktivitas mencurigakan seperti scanning, brute-force login, dan serangan terhadap layanan web.

Dengan memonitor log Apache menggunakan Wazuh dan machine learning, sistem dapat mendeteksi anomali keamanan pada web server secara real-time dan memberikan notifikasi kepada administrator.

2.2.4 Wazuh

Wazuh merupakan *platform open source* yang berfungsi sebagai sistem deteksi ancaman, pemantauan keamanan dan respons insiden. *Platform Wazuh* merupakan implementasi dari *Security Information and Event Management (SIEM)*. Wazuh menyediakan berbagai fitur yang dapat menganalisis data log, intruksi dan deteksi malware, pemantauan integritas file, penilaian konfigurasi dan deteksi kerentanan system [11]. Wazuh memiliki 2 buah komponen yaitu:

a. Wazuh Agent

Wazuh Agent merupakan sebuah end points seperti desktop, server, instans cloud atau mesin virtual, yang dapat melakukan pencegahan, deteksi, dan respons.

b. Wazuh Server

Wazuh Server merupakan server yang bertugas menganalisis data yang diterima oleh *Agent* dan memprosesnya melalui decoder dan aturan.

Wazuh Agent akan mengirimkan log yang didapatkan ke Wazuh Server untuk dilakukan analisis dan deteksi ancaman. Sebelum itu, Wazuh Agent akan membuat sebuah

koneksi dengan layanan server. Kemudian Wazuh Server akan menerjemahkan log yang diterima menggunakan analysis engine. Log yang terdeteksi sebagai ancaman akan dibuatkan suatu Alert yang menyimpan rule id dan rule name. Kemudian log tersebut akan ditampung ke dalam penyimpanan Wazuh. Kemudian mengirimkan Alert dan log akan divisualisasikan informasi ke Interface dari Wazuh dashboard.



Gambar 2. 1 Wazuh

2.2.5 AWS

Amazon Web Service menyediakan layanan infrastruktur kunci bisnis dunia yang bermanfaat untuk membangun bisnis perusahaan dan sebagai akses penawaran produk dari suatu perusahaan ke perusahaan lain. Amazon Web Services juga menyediakan layanan cloud computing sehingga pengguna dapat menyimpan data secara permanen di dalam server di internet. masing-masing instance secara logika dapat saling berbagi waktu kerja CPU dan berbagi ruang memori dengan cara yang benar tanpa saling berinterferensi satu dengan lainnya [14].

AWS juga memiliki fungsionalitas terdalem dalam semua layananannya. Misalnya, AWS menawarkan ragam basis data paling luas yang dibangun untuk tujuan beragam jenis aplikasi sehingga Anda dapat memilih alat yang tepat untuk tugas tersebut untuk mendapatkan biaya dan performa terbaik.



Gambar 2. 2 Amazon Web Service

2.2.6 Elasticsearch

Elasticsearch merupakan salah satu database yang masuk ke dunia NoSQL dengan fokus di search engine database. Elasticsearch ditenagai oleh Apache Lucene yang juga merupakan search engine database yang memiliki query low level. Elasticsearch memiliki query yang lebih mudah untuk digunakan karena berbasis RESTful.

Elasticsearch memiliki konsep yang cukup unik. Dimana dapat mengasumsikan indeks sebagai "database", types sebagai "tabel" dan dokumen sebagai record atau row. Sedangkan mapping dapat diasumsikan sebagai "skema tabel". Di Elasticsearch tidak ada transaction dan dapat membuat struktur indeks tergantung dengan kebutuhan kita. Selain itu dapat diatur untuk menjadi sebuah sistem terdistribusi terhadap sejumlah server [15].



Gambar 2. 3 Elasticsearch

2.2.7 Kibana

Kibana merupakan aplikasi visualisasi data open-source yang terintegrasi dengan Elasticsearch dan digunakan untuk menampilkan serta menganalisis data dalam bentuk grafik, tabel, maupun dashboard interaktif. Kibana memungkinkan pengguna melakukan pencarian dan filtering data log secara real-time

berdasarkan waktu, tingkat severity, maupun field tertentu yang tersimpan dalam indeks Elasticsearch.

Dalam arsitektur sistem yang dikembangkan pada penelitian ini, Kibana digunakan sebagai antarmuka monitoring untuk membaca index `wazuh-alerts-*` yang tersimpan di Elasticsearch. Dengan menggunakan fitur Discover dan Dashboard, administrator dapat melihat aktivitas log berdasarkan rentang waktu tertentu melalui field `@timestamp`, serta melakukan analisis pola serangan berdasarkan tingkat rule dan agent sumber log.

Penggunaan Kibana dalam sistem ini bertujuan untuk memisahkan fungsi penyimpanan log dan fungsi visualisasi, sehingga monitoring dapat dilakukan secara lebih fleksibel dan efisien dibandingkan penyimpanan berbasis file lokal.



Gambar 2. 4 Kibana

2.2.8 Machine Learning

Machine Learning (ML) merupakan cabang dari kecerdasan buatan (Artificial Intelligence) yang memungkinkan sistem komputer untuk mempelajari pola dari data dan membuat keputusan atau prediksi tanpa diprogram secara eksplisit untuk setiap kondisi [16]. Berbeda dengan pendekatan tradisional berbasis aturan (rule-based), machine learning memanfaatkan data historis untuk membangun model yang mampu melakukan generalisasi terhadap data baru.

Secara umum, machine learning dibagi menjadi tiga pendekatan utama, yaitu supervised learning, unsupervised learning,

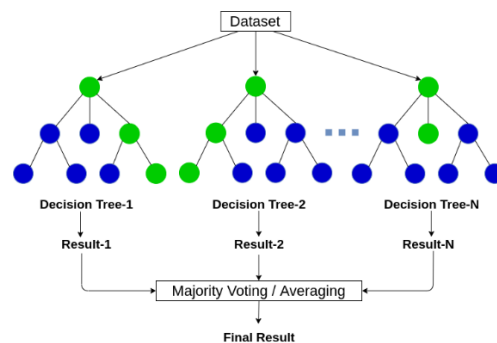
dan reinforcement learning. Pada penelitian ini digunakan pendekatan supervised learning, dimana model dilatih menggunakan dataset yang telah memiliki label kelas. Label tersebut merepresentasikan kategori tertentu yang ingin diprediksi oleh model.

Dalam konteks penelitian ini, klasifikasi dilakukan terhadap data log keamanan yang dihasilkan oleh sistem Wazuh. Data log dikategorikan ke dalam dua kelas, yaitu:

1. 0 = (Normal)
2. 1 = (Anomali)

Label diberikan berdasarkan karakteristik tertentu dari log, seperti tingkat severity rule (rule_level) dan jumlah kemunculan alert (firedtimes). Dengan adanya label tersebut, model dapat mempelajari perbedaan pola antara aktivitas normal dan aktivitas yang berpotensi sebagai serangan.

1. Algoritma Random Forest



Gambar 2. 5 Algoritma Random Forest

Algoritma yang digunakan dalam penelitian ini adalah Random Forest, yang diperkenalkan oleh Leo Breiman pada tahun 2001 [5]. Random Forest merupakan metode ensemble learning yang menggabungkan sejumlah pohon keputusan (decision tree) untuk meningkatkan performa klasifikasi dan mengurangi risiko overfitting.

Random Forest bekerja menggunakan dua prinsip utama, yaitu:

a. Bootstrap Aggregating (Bagging).

Dataset training diambil secara acak dengan teknik bootstrap sampling (sampling dengan pengembalian). Setiap pohon keputusan dilatih menggunakan subset data yang berbeda.

b. Random Feature Selection

Pada setiap node dalam pohon keputusan, hanya sebagian fitur yang dipilih secara acak untuk menentukan pemisahan (split) terbaik. Pendekatan ini bertujuan untuk mengurangi korelasi antar pohon serta meningkatkan variasi model.

Setelah seluruh pohon dibangun, hasil prediksi akhir ditentukan melalui mekanisme majority voting, yaitu kelas yang paling banyak dipilih oleh seluruh pohon akan menjadi hasil klasifikasi akhir. Pendekatan ini membuat Random Forest memiliki beberapa keunggulan, antara lain:

- a. Mampu menangani data berdimensi tinggi.
- b. Stabil terhadap data yang tidak seimbang.
- c. Tidak mudah mengalami overfitting.
- d. Mampu membentuk batas keputusan non-linear.
- e. Relatif ringan secara komputasi dibanding metode deep learning.

2. Relevansi Random Forest dalam Deteksi Anomali Log

Data log keamanan memiliki karakteristik yang kompleks dan tidak selalu mengikuti pola linier. Sebagai contoh, aktivitas login SSH pada jam kerja dapat dianggap normal, tetapi aktivitas dengan frekuensi tinggi pada waktu tertentu dapat mengindikasikan percobaan brute-force. Oleh karena itu, diperlukan algoritma yang mampu mempelajari kombinasi

beberapa fitur secara simultan. Dalam penelitian ini digunakan fitur seperti:

- a. firedtimes (Jumlah kemunculan alert)
- b. rule_level (Tingkat severity rule)
- c. hour (Jam kejadian)
- d. is_night (Indikator kejadian pada jam 00.00–05.00)
- e. is_ssh (Indikator aktivitas SSH)
- f. log_length (Panjang karakter log)

3. Evaluasi Model Machine Learning

Untuk mengukur performa model klasifikasi, digunakan confusion matrix, yang terdiri dari:

- a. True Positive (TP)
- b. True Negative (TN)
- c. False Positive (FP)
- d. False Negative (FN)

Berdasarkan nilai tersebut dihitung beberapa metrik evaluasi, yaitu:

- a. Accuracy = Mengukur tingkat ketepatan prediksi secara keseluruhan.
- b. Precision = Mengukur proporsi prediksi anomali yang benar-benar anomali.
- c. Recall = Mengukur kemampuan model dalam mendeteksi seluruh data anomali.
- d. F1-Score = Nilai keseimbangan antara precision dan recall.

Dalam sistem keamanan, nilai recall menjadi metrik yang sangat penting karena berkaitan langsung dengan kemampuan sistem dalam mendeteksi seluruh aktivitas serangan. False Negative (serangan yang tidak terdeteksi) memiliki dampak yang lebih berbahaya dibanding False Positive.

4. Alasan Pemilihan Random Forest

Walaupun metode deep learning seperti LSTM atau Autoencoder banyak digunakan dalam deteksi anomali log, penelitian ini memilih Random Forest dengan pertimbangan sebagai berikut:

- a. Kompleksitas implementasi lebih sederhana
- b. Kebutuhan komputasi lebih ringan
- c. Mudah diintegrasikan dengan sistem SIEM berbasis Wazuh
- d. Sesuai dengan skala penelitian tugas akhir
- e. Memberikan interpretasi model yang lebih jelas dibanding model deep learning

Dengan demikian, machine learning dalam penelitian ini berfungsi sebagai lapisan analisis tambahan di atas sistem rule-based Wazuh. Integrasi ini memungkinkan sistem tidak hanya mendeteksi berdasarkan kecocokan rule statis, tetapi juga mempertimbangkan pola historis data log untuk meningkatkan kualitas klasifikasi aktivitas keamanan.

2.2.9 Python

Python adalah bahasa pemrograman populer yang dapat digunakan untuk berbagai hal, mulai dari membangun website, menganalisis data, scripting, hingga pembuatan game yang bersifat open source, sehingga dapat menggunakannya secara gratis.

Python merupakan bahasa pemrograman komputer yang biasa dipakai untuk membangun situs, software/aplikasi, mengotomatiskan tugas dan melakukan analisis data. Bahasa pemrograman ini termasuk bahasa tujuan umum [17]. Artinya, ia bisa digunakan untuk membuat berbagai program berbeda, bukan khusus untuk masalah tertentu saja. Karena sifatnya yang serba guna dan mudah digunakan, ia menjadi bahasa pemrograman yang paling banyak digunakan. Terutama untuk mereka yang masih pemula.



Gambar 2. 6 Python

2.2.10 Telegram Bot

Telegram merupakan salah satu platform media komunikasi yang sudah ada sejak tahun 2013 silam dan masih eksis digunakan hingga sekarang. Salah satu fitur yang ada pada telegram adalah adanya 20 Bot API yang dapat digunakan pengguna sebagai asisten atau fitur otomatisasi pada akun telegram. Telegram Bot Application Programming Interface (API) adalah sebuah teknologi *open source* yang disediakan oleh Telegram untuk membangun aplikasi bot Telegram bagi para pengembang. Bot API ini merupakan interface berbasis HTTP untuk menghubungkan bot yang dikembangkan oleh para pengembang dengan sistem Telegram. Bot Telegram merupakan sebuah akun khusus yang tidak memerlukan nomor telepon [18]. Akun ini berfungsi sebagai interface untuk menjalankan code yang sudah dibangun.

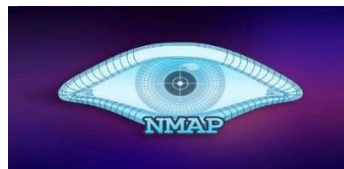


Gambar 2. 7 Telegram

2.2.11 Nmap

NMAP adalah singkatan dari Network Mapper yang merupakan sebuah tool atau alat yang bersifat open source. Alat ini hanya digunakan secara khusus untuk eksplorasi jaringan serta melakukan audit terhadap keamanan dari jaringan. NMAP bekerja dengan mengirimkan paket menuju target dengan menggunakan IP

raw yang memiliki cara kerja yang canggih sehingga bisa menentukan mana saja host yang aktif. Selain itu, NMAP juga melaksanakan bruteforce menuju port list yang dimilikinya yang nantinya akan dipakai di dalam menentukan status dari port host yang sedang aktif, baik itu dengan cara melakukan filter, close, maupun juga open [19]. NMAP juga diketahui mampu melihat sistem operasi apa dan lengkap dengan versinya yang digunakan atau terinstal pada suatu perangkat.



Gambar 2. 8 NMAP

2.2.12 Metasploit

Metasploit adalah sebuah proyek keamanan komputer yang dapat menyediakan informasi tentang kerentanan keamanan dan membantu dalam pengujian penetrasi dan pengembangan signature IDS. Metasploit memiliki beberapa sub proyek, sub proyek yang paling terkenal adalah open source, Metasploit Framework, alat untuk pengembangan dan alat pengeksekusi kode eksploitasi terhadap mesin target jarak jauh. sub proyek lainnya termasuk database opcode, arsip shellcode, dan penelitian terkait lainnya. Metasploit terkenal dengan anti-forensik dan evasion tools, yang beberapa di antaranya di bangun ke dalam Metasploit Framework [20].



Gambar 2. 9 Metasploit

BAB III

ANALISIS DAN PERANCANGAN

3.1 Analisis Sistem yang Sedang Berjalan

Sistem pemantauan keamanan web server yang berjalan saat ini masih bersifat reaktif dan bergantung pada proses analisis manual terhadap log aktivitas. Administrator umumnya hanya menggunakan fitur bawaan sistem operasi, seperti auth.log, syslog, atau apache2.log, untuk memeriksa aktivitas mencurigakan. Pendekatan ini memakan waktu dan rentan terhadap kesalahan manusia, terutama jika jumlah log yang dikumpulkan sangat besar.

Selain itu, sistem belum memiliki kemampuan otomatis dalam mendeteksi anomali atau aktivitas abnormal yang berpotensi sebagai ancaman siber. Proses monitoring juga belum terintegrasi dengan sistem notifikasi real-time, sehingga potensi serangan sering terlambat ditangani. Oleh karena itu, diperlukan sistem Security Information and Event Management (SIEM) berbasis Wazuh yang mampu mengumpulkan, mengkorelasikan, dan menganalisis data log secara otomatis, serta ditingkatkan dengan integrasi machine learning (Random Forest) untuk mendeteksi anomali secara cerdas.

3.1.1 Analisis Kebutuhan Perangkat Keras

Berikut Spesifikasi hardware yang diperlukan dalam implementasi tugas akhir ini:

1. Spesifikasi Instance Wazuh Manager + Dashboard

Tabel 3. 1 Spesifikasi Instance Wazuh Manager + Dashboard

Spesifikasi	Keterangan
Instance Type	t2.medium
Processor	2 vCPU
RAM	4 GB
Storage	40 GB SSD
Sistem Operasi	Ubuntu Server 22.04
Role	Wazuh Manager & Dashboard

2. Spesifikasi Instance Elasticsearch

Tabel 3. 2 Spesifikasi Instance Elasticsearch

Spesifikasi	Keterangan
Instance Type	t3.medium
Processor	4 vCPU
RAM	8 GB
Storage	80 GB SSD
Sistem Operasi	Ubuntu Server 22.04
Role	Penyimpanan log & indexing untuk SIEM

3. Spesifikasi Instance Wazuh Agent (Web Server)

Tabel 3. 3 Spesifikasi Instance Wazuh Agent (Web Server)

Spesifikasi	Keterangan
Instance Type	t3.micro

Processor	2 vCPU
RAM	1–2 GB
Storage	20 GB
Role	Web Server + Wazuh Agent

4. Spesifikasi Instance Machine Learning Engine

Tabel 3. 4 Spesifikasi Instance Machine Learning Engine

Spesifikasi	Keterangan
Instance Type	t3.medium
Processor	2 vCPU
RAM	8 GB
Storage	30 GB
Role	Training model Random Forest & inferensi anomali

3.1.2 Analisis Kebutuhan Perangkat Lunak

Berikut ini kebutuhan perangkat lunak yang diperlukan dalam implementasi tugas akhir ini:

Tabel 3.5 Analisis kebutuhan perangkat lunak

Nama	Versi	Fungsi
Ubuntu Server	22.04 LTS	Sistem operasi utama pada seluruh instance AWS
Wazuh Manager	v4.x	Mengelola agent, mendeteksi event, rule-based alert
Wazuh Agent	v4.x	Mengirim log dari Web

		Server ke Wazuh Manager
Wazuh Dashboard	v4.x	Visualisasi alert & monitoring keamanan
Elasticsearch	8.x	Penyimpanan log & indexing untuk SIEM
Python	3.10+	ML Engine: preprocessing, training, inferensi
Scikit Learn	1.x	Algoritma Random Forest untuk klasifikasi anomali
Pandas	2.x	Pemrosesan dataset log
Numpy	1.x	Operasi matematis untuk ML
Joblib	1.x	Menyimpan model ML
Requests	2.x	Mengambil log melalui REST API Elasticsearch
Hydra	Terbaru	Simulasi brute-force untuk data uji
Nmap	Terbaru	Simulasi scanning
Telegram Bot API	-	Notifikasi keamanan real-time

Tabel 3.6 Layanan AWS yang digunakan

Layanan	Fungsi
EC2	Menjalankan Wazuh Manager, Agent, Elasticsearch, dan ML Engine
VPC	Mengatur jaringan dan segmentasi keamanan
Security Group	Firewall & kontrol akses
Elastic IP	IP statis untuk server penting
Telegram API	Sistem notifikasi

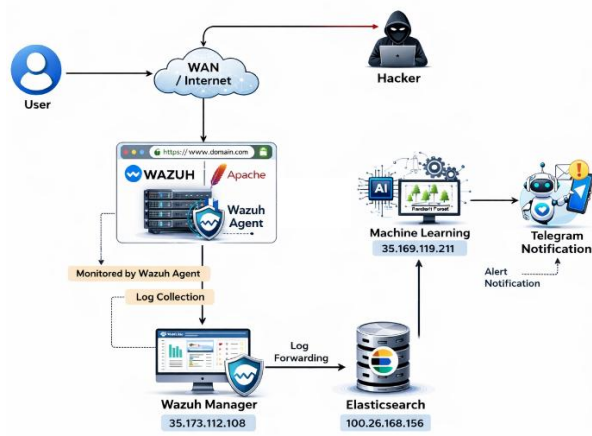
IAM	Manajemen akses & credential
CloudWatch	Monitoring instance

3.2 Rancangan Sistem

Rancangan sistem merupakan tahap penting dalam pengembangan sistem SIEM berbasis Wazuh yang terintegrasi dengan algoritma machine learning. Tahap ini menghasilkan gambaran alur kerja sistem, interaksi antar komponen, serta proses pendeteksian anomali mulai dari pengumpulan log hingga notifikasi akhir. Rancangan dibuat untuk memastikan setiap komponen menjalankan fungsi secara optimal serta komunikasi antar komponen berjalan efisien dan real-time.

3.2.1 Rancangan Topologi

Berikut ini merupakan gambar dari rancangan topologi tugas akhir untuk keamanan web server:



Gambar 3.1 Rancangan Topologi

Pada gambar 3.1 terlihat bahwa sistem terdiri dari empat instance utama pada AWS EC2, yaitu Wazuh Manager, Wazuh Agent (Web Server), Elasticsearch, dan Machine Learning Engine. Setiap instance memiliki peran berbeda dan saling terhubung melalui jaringan VPC yang sama.

Pada bagian awal, Wazuh Agent yang terpasang pada Web Server bertugas mengumpulkan berbagai log aktivitas seperti auth.log, syslog, dan apache2.log. Log yang dikumpulkan dikirimkan secara real-time ke Wazuh Manager melalui koneksi terenkripsi. Wazuh Manager kemudian melakukan proses decoder parsing dan rule matching untuk mengidentifikasi pola ancaman yang sesuai dengan rule yang telah ditentukan.

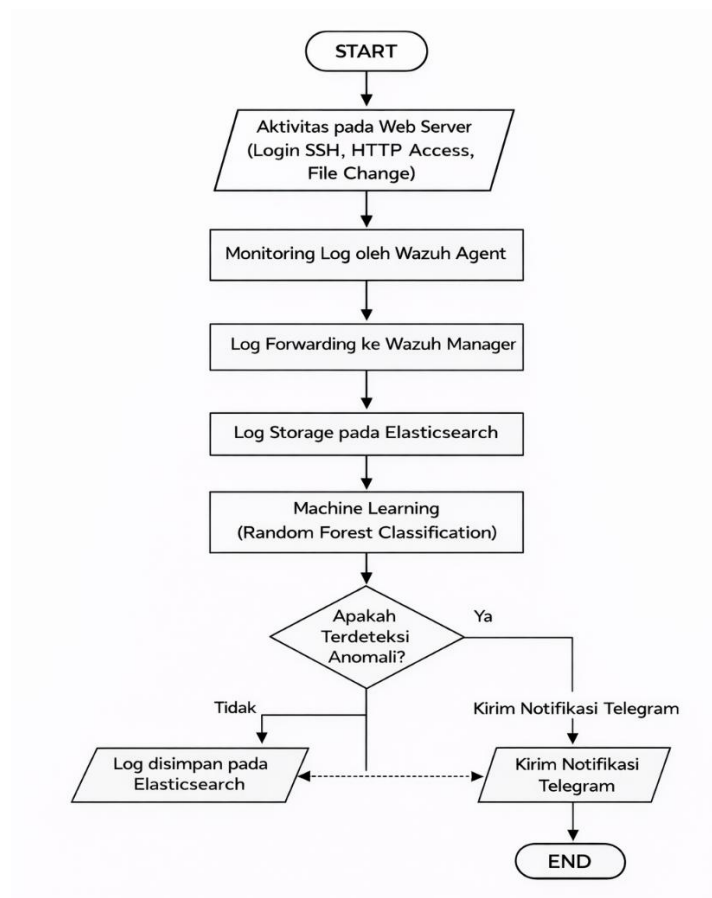
Selanjutnya, seluruh log yang telah diproses dikirimkan menuju Elasticsearch sebagai pusat penyimpanan data log terstruktur. Elasticsearch berfungsi melakukan indexing terhadap log sehingga dapat diakses untuk analisis lebih lanjut, visualisasi, serta pembentukan dataset untuk proses pembelajaran mesin.

Komponen Machine Learning Engine mengambil log dari Elasticsearch melalui REST API untuk dilakukan preprocessing dan klasifikasi menggunakan model Random Forest. Model ini mendeteksi adanya aktivitas anomali yang tidak teridentifikasi oleh mekanisme rule-based. Jika terdeteksi anomali, sistem akan meneruskan hasil klasifikasi ke Telegram Notification Service untuk mengirimkan peringatan kepada administrator secara otomatis.

Secara keseluruhan, rancangan topologi ini mendukung proses monitoring keamanan yang bersifat real-time, terpusat, dan adaptif, serta mampu meningkatkan kemampuan deteksi ancaman melalui integrasi metode rule-based dan machine learning.

3.2.2 Rancangan Flowchart

Berikut rancangan flowchart dari alur kerja dari layanan AWS yang dibangun untuk meningkatkan ketersediaan dan keamanan web server:



Gambar 3. 2 Rancangan Flowchart

Flowchart pada Gambar X menjelaskan alur kerja sistem monitoring keamanan web server menggunakan Wazuh dan Machine Learning. Proses dimulai dari aktivitas pada web server, seperti login SSH, akses HTTP, dan perubahan file (file change). Aktivitas tersebut menghasilkan log yang menjadi input utama sistem.

Selanjutnya, Wazuh Agent melakukan proses monitoring dan pengumpulan log pada web server. Log yang terkumpul kemudian diteruskan (log forwarding) ke Wazuh Manager untuk diproses lebih lanjut. Setelah itu, log disimpan pada Elasticsearch sebagai pusat penyimpanan (log storage) agar data dapat diindeks dan digunakan kembali untuk kebutuhan monitoring maupun analisis lanjutan.

Tahap berikutnya adalah proses Machine Learning (Random Forest Classification). Pada tahap ini, Machine Learning membaca log dari Elasticsearch dan melakukan klasifikasi untuk menentukan apakah log tersebut termasuk aktivitas normal atau anomali. Sistem kemudian masuk ke tahap keputusan “Apakah terdeteksi anomali?”.

1. Jika anomali terdeteksi (Ya), sistem akan mengirim notifikasi Telegram kepada administrator sebagai peringatan dini, kemudian proses berakhir.
2. Jika anomali tidak terdeteksi (Tidak), maka log hanya tetap tersimpan di Elasticsearch sebagai arsip monitoring dan tidak mengirim notifikasi, lalu proses berakhir.

Dengan alur tersebut, sistem memastikan seluruh log tercatat di Elasticsearch, sementara notifikasi Telegram hanya dikirim ketika terdeteksi adanya aktivitas yang terindikasi anomali.

BAB IV

IMPLEMENTASI DAN PENGUNJIAN

4.1 Implementasi

Bab ini membahas proses implementasi sistem *Security Information and Event Management (SIEM)* berbasis Wazuh yang digunakan untuk melakukan pemantauan dan deteksi anomali keamanan pada web server. Implementasi sistem dilakukan berdasarkan perancangan yang telah dijelaskan pada Bab III. Pada tahap ini, sistem benar-benar dibangun dan diuji secara langsung pada lingkungan cloud menggunakan *Amazon Web Services (AWS)*. Sistem diimplementasikan menggunakan empat buah instance AWS EC2 yang masing-masing memiliki fungsi berbeda, yaitu sebagai Wazuh Manager dan Dashboard, Web Server dengan Wazuh Agent, Elasticsearch, serta instance khusus untuk pengolahan dan analisis Machine Learning. Pembagian ini dilakukan agar setiap komponen dapat berjalan secara optimal dan sesuai dengan perannya.

4.2 Arsitektur Infrastruktur AWS

Arsitektur infrastruktur sistem dibangun pada satu Virtual Private Cloud (VPC) AWS agar seluruh instance dapat saling berkomunikasi secara aman. Setiap instance dikonfigurasi menggunakan *security group* untuk membatasi akses jaringan sesuai dengan kebutuhan layanan.

4.2.1 Wazuh Manager dan Wazuh Dashboard

Instance ini berfungsi sebagai pusat pengelolaan sistem SIEM. Wazuh Manager bertugas menerima log dari Wazuh Agent, melakukan proses decoding, rule matching, serta menghasilkan alert keamanan. Pada arsitektur final sistem, Wazuh Dashboard tidak digunakan sebagai media monitoring utama. Seluruh

visualisasi log dilakukan menggunakan Kibana yang terintegrasi langsung dengan Elasticsearch.

4.2.2 Web Server dan Wazuh Agent

Instance ini berperan sebagai web server berbasis Ubuntu Server yang menjadi objek pemantauan keamanan. Pada server ini dipasang Wazuh Agent yang bertugas mengumpulkan log aktivitas sistem, seperti `auth.log`, `syslog`, dan `apache2.log`, kemudian mengirimkannya ke Wazuh Manager secara *real-time*.

4.2.3 Elasticsearch

Instance Elasticsearch digunakan sebagai media penyimpanan dan pengindeksan data log. Log yang dikirimkan dari Wazuh Manager disimpan dalam bentuk indeks sehingga dapat dicari, dianalisis, serta digunakan sebagai sumber data untuk proses *machine learning*.

4.2.4 Machine Learning

Instance Machine Learning Engine digunakan untuk proses analisis berbasis *machine learning*. Data log yang tersimpan di Elasticsearch diambil melalui REST API untuk dilakukan preprocessing, ekstraksi fitur, pelatihan model Random Forest, serta proses inferensi untuk mendeteksi anomali keamanan.

4.3 Pembuatan dan Konfigurasi AWS

4.3.1 Instance Wazuh Manager

1. Launch Instance EC2



Gambar 4. 1 Launch Instance pada AWS EC2

Berdasarkan gambar 4.1, proses pembuatan server dimulai dengan memilih menu Launch Instance pada halaman EC2.

Langkah ini merupakan tahap awal untuk membuat mesin virtual yang nantinya akan dikonfigurasi sebagai Wazuh Manager.

2. Memberi Name and Tags

Name and tags [Info](#)

Name

WazuhManager

Gambar 4. 2 Konfigurasi Name and Tags Instance

Pada tahap ini dilakukan pemberian nama instance dengan nama “WazuhManager”. Pemberian nama ini bertujuan agar server mudah dikenali pada dashboard AWS, terutama jika terdapat beberapa instance yang berjalan dalam satu akun.

3. Memilih Amazon Machine Image (AMI)

Quick Start

Amazon Linux aws	macOS Mac	Ubuntu ubuntu®	Windows Microsoft	Red Hat Red Hat	SUSE Linux SUSE	Debian debian
---------------------	--------------	-------------------	----------------------	--------------------	--------------------	------------------

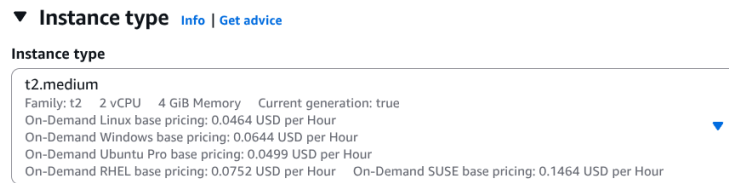
Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
 ami-0b6c6ebd2801a5cb (64-bit (x86)) / ami-0071c8c431eea0edb (64-bit (Arm))
 Virtualization: hvm ENA enabled: true Root device type: ebs

Gambar 4. 3 Pemilihan Amazon Machine Image (AMI)

Pada tahap ini dipilih Ubuntu Server 22.04 LTS sebagai sistem operasi. Ubuntu dipilih karena stabil, ringan, dan banyak digunakan untuk server. Selain itu, sistem operasi ini juga mendukung instalasi Wazuh Manager dengan baik.

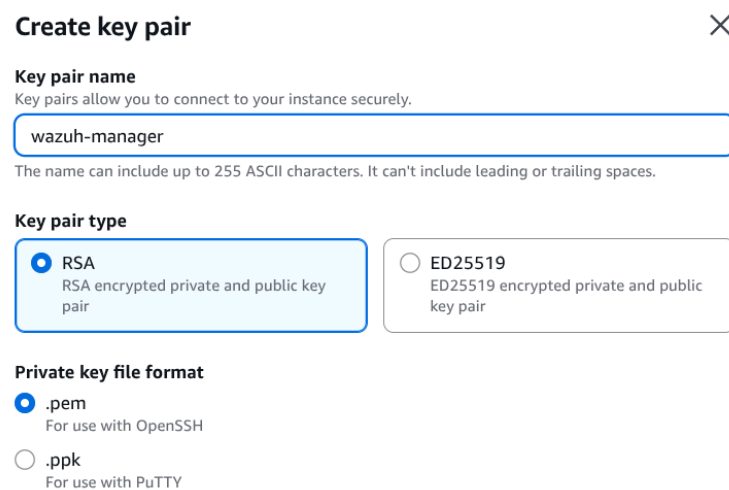
4. Memilih Tipe Instance



Gambar 4. 4 Pemilihan Tipe Instance

Tipe instance disesuaikan dengan kebutuhan sistem. Pada implementasi ini digunakan tipe t2.medium dengan spesifikasi 2 vCPU dan 4 GB RAM. Spesifikasi ini cukup untuk menjalankan Wazuh Manager dan menerima data log dari beberapa agent.

5. Membuat Key Pair



Gambar 4. 5 Pembuatan Key Pair

Key Pair digunakan untuk mengakses server melalui SSH. Pada tahap ini dibuat file dengan format .pem yang nantinya digunakan saat melakukan koneksi ke server menggunakan terminal. Penggunaan key pair lebih aman dibandingkan login menggunakan password.

6. Memasukkan Jumlah Penyimpanan



Gambar 4. 6 Konfigurasi Storage (EBS)

Pada bagian ini ditentukan kapasitas penyimpanan sebesar 40 GB menggunakan Elastic Block Store (EBS). Penyimpanan ini digunakan untuk sistem operasi dan menyimpan data log dari Wazuh Manager.

7. Mengaitkan dengan Elastic IP

Summary			
Allocated IPv4 address 52.71.135.122	Type Public IP	Allocation ID eipalloc-0869da680057c4080	Reverse DNS record -
Association ID eipassoc-046c4d22ab78d4941	Scope VPC	Associated instance ID i-0eafe6835bf77c592	Private IP address 172.31.22.249
Network interface ID eni-0fcbdac218597bec3	Network interface owner account ID 990220143794	Public DNS ec2-52-71-135-122.compute-1.amazonaws.com	NAT Gateway ID -
Address pool Amazon	Network border group us-east-1	Service managed -	

Gambar 4. 7 Konfigurasi Elastic IP pada AWS

Pada tahap ini dilakukan proses Allocate Elastic IP Address, kemudian IP tersebut dihubungkan (associate) ke instance Wazuh Manager. Elastic IP digunakan agar alamat server tidak berubah meskipun instance di-restart. Hal ini penting agar agent tetap bisa terhubung ke server dengan alamat IP yang sama.

8. Konfigurasi Security Group dan port

Security group rule ID	IP version	Type	Protocol	Port range	Source
sg-012a5e7024283c434	IPv4	HTTP	TCP	80	0.0.0.0/0
sg-0b6027e782d957b07	IPv4	Custom TCP	TCP	1515	0.0.0.0/0
sg-03f0ac7657468ed6e	IPv4	HTTPS	TCP	443	0.0.0.0/0
sg-048478799fa40d6ee	IPv4	Custom TCP	TCP	5601	0.0.0.0/0
sg-097d04300ca189ab7	IPv4	SSH	TCP	22	0.0.0.0/0
sg-066395ceaa863397	IPv4	Custom TCP	TCP	1514	0.0.0.0/0

Gambar 4. 8 Konfigurasi Security Group dan Port

Pada tahap ini dilakukan konfigurasi Security Group untuk membuka port yang dibutuhkan, yaitu:

- e. Port 22: Digunakan untuk akses SSH (remote server).
- f. Port 80: Digunakan untuk akses HTTP.
- g. Port 443: Digunakan untuk akses HTTPS yang lebih aman.
- h. Port 1514/1515: Digunakan untuk komunikasi antara Wazuh Agent dan Wazuh Manager.
- i. Port 5601: Digunakan untuk mengakses dashboard monitoring melalui web.

4.3.2 Instance Wazuh Agent (WEB Server)

1. Launch Instance EC2



Gambar 4. 9 Launch Instance pada AWS EC2

Pembuatan instance dimulai dengan memilih menu Launch Instance pada layanan Amazon EC2 di Amazon Web Services. Instance ini dibuat terpisah dari Wazuh Manager agar pembagian beban kerja lebih optimal.

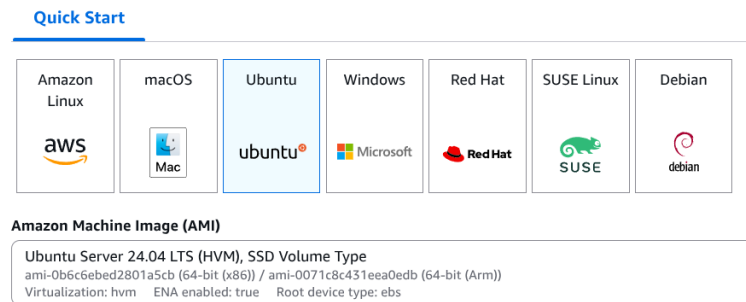
2. Memberi Name and Tags



Gambar 4. 10 Konfigurasi Name and Tags Wazuh Agent (Web Server)

Instance diberi nama “WazuhAgent” agar mudah dibedakan dengan server Wazuh Manager. Penamaan ini penting agar tidak terjadi kesalahan saat melakukan konfigurasi atau pengelolaan server.

3. Memilih Amazon Machine Image (AMI)



Gambar 4. 11 Pemilihan AMI Wazuh Agent (Web Server)

Pada tahap ini dipilih Ubuntu Server 22.04 LTS sebagai sistem operasi. Ubuntu dipilih karena stabil, ringan, dan banyak digunakan untuk server. Selain itu, sistem operasi ini juga mendukung instalasi Wazuh Manager dengan baik.

4. Memilih Tipe Instance



Gambar 4. 12 Pemilihan Tipe Instance Wazuh Agent (Web Server)

Untuk Wazuh Agent, spesifikasi yang digunakan lebih ringan dibandingkan Manager. Instance yang digunakan dapat berupa t3.micro atau setara, karena agent hanya bertugas mengirimkan log ke server pusat dan tidak melakukan analisis berat.

5. Membuat Key Pair

Create key pair ✕

Key pair name
Key pairs allow you to connect to your instance securely.

wazuh-agent

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ **RSA**
RSA encrypted private and public key pair

☐ **ED25519**
ED25519 encrypted private and public key pair

Private key file format

☒ **.pem**
For use with OpenSSH

☐ **.ppk**
For use with PuTTY

Gambar 4. 13 Konfigurasi Key Pair Wazuh Agent (Web Server)

Pada tahap ini digunakan key pair yang sudah dibuat sebelumnya agar dapat mengakses server melalui SSH. Dengan menggunakan key pair yang sama, proses manajemen server menjadi lebih mudah.

6. Memasukkan Jumlah Penyimpanan

▼ **Configure storage** [Info](#)

1x 20 GiB gp3 Root volume, 3000 IOPS, Not encrypted

Gambar 4. 14 Konfigurasi Storage Wazuh Agent (Web Server)

Kapasitas penyimpanan yang digunakan pada instance agent lebih kecil, misalnya 20 GB, karena hanya digunakan untuk sistem operasi dan penyimpanan log sementara sebelum dikirim ke Wazuh Manager.

7. Mengaitkan dengan Elastic IP

Summary			
Allocated IPv4 address 44.205.185.179	Type Public IP	Allocation ID eipalloc-0c2095b6a77c0a9f2	Reverse DNS record -
Association ID eipassoc-0c8f654b342ce9b69	Scope VPC	Associated instance ID i-0e7cfc66047ae593a	Private IP address 172.31.19.160
Network interface ID eni-0027bb86c2ba8936db	Network interface owner account ID 590220143794	Public DNS ec2-44-205-185-179.compute-1.amazonaws.com	NAT Gateway ID -
Address pool Amazon	Network border group us-east-1	Service managed -	

Gambar 4. 15 Konfigurasi Elastic IP Wazuh Agent (Web Server)

Pada tahap ini dilakukan proses Allocate Elastic IP Address, kemudian IP tersebut dihubungkan (associate) ke instance Wazuh Agent. Elastic IP digunakan agar alamat server tidak berubah saat instance di-restart, sehingga memudahkan proses koneksi dan pengujian sistem.

8. Konfigurasi Security Group dan port

Security group rule ID	IP version	Type	Protocol	Port range	Source
sgr-035a79add0b139127	IPv4	HTTP	TCP	80	0.0.0.0/0
sgr-0b6aabb05e6be3b3bc	IPv4	HTTPS	TCP	443	0.0.0.0/0
sgr-012663fc10fb20c8a	IPv4	SSH	TCP	22	0.0.0.0/0

Gambar 4. 16 Konfigurasi Security Group Wazuh Agent (Web Server)

Pada tahap ini dilakukan konfigurasi Security Group untuk membuka port yang dibutuhkan, yaitu:

- Port 22: Digunakan untuk akses SSH (remote server).
- Port 80: Digunakan untuk akses HTTP.
- Port 443: Digunakan untuk akses HTTPS yang lebih aman.

4.3.3 Instance Elasticsearch

1. Launch Instance EC2



Gambar 4. 17 Launch Instance

Pembuatan instance dimulai dengan memilih menu Launch Instance pada layanan Amazon EC2 di Amazon Web Services. Instance ini dibuat terpisah dari Wazuh Manager agar pembagian beban kerja lebih optimal.

2. Memberi Name and Tags

Name and tags [Info](#)

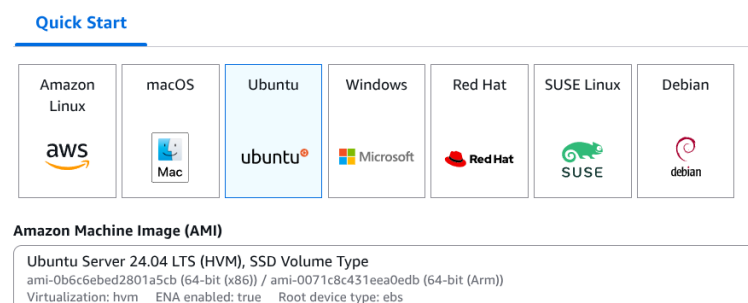
Name

Elasticsearch

Gambar 4. 18 Konfigurasi Name and Tags Elasticsearch

Instance diberi nama “ElasticsearchServer” agar mudah dikenali pada dashboard AWS. Penamaan ini penting untuk membedakan server Elasticsearch dengan server lainnya seperti Wazuh Manager dan Wazuh Agent.

3. Memilih Amazon Machine Image (AMI)



Gambar 4. 19 Pemilihan AMI Elasticsearch

Pada tahap ini dipilih Ubuntu Server 22.04 LTS sebagai sistem operasi. Ubuntu dipilih karena stabil dan kompatibel dengan instalasi Elasticsearch serta mudah dalam proses konfigurasi.

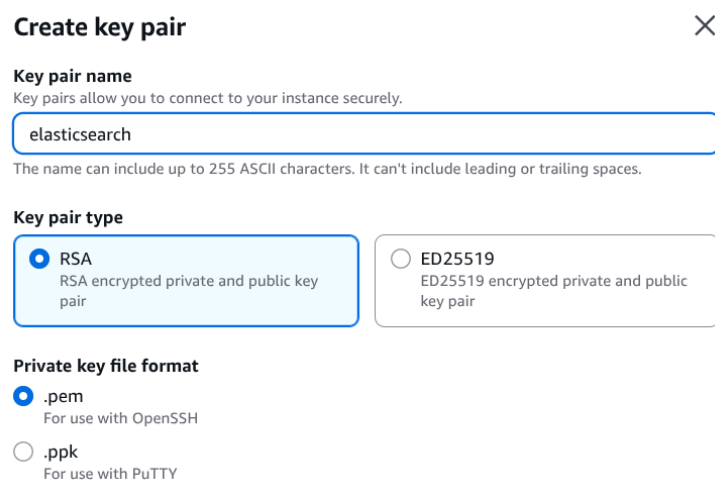
4. Memilih Tipe Instance



Gambar 4. 20 Pemilihan Tipe Instance Elasticsearch

Untuk Elasticsearch digunakan spesifikasi yang lebih tinggi dibandingkan agent, yaitu t3.medium atau setara (2 vCPU dan 4 GB RAM). Hal ini karena Elasticsearch bertugas menyimpan dan memproses data log dalam jumlah besar, sehingga membutuhkan sumber daya yang cukup.

5. Membuat Key Pair



Gambar 4. 21 Konfigurasi Key Pair Elasticsearch

Pada tahap ini digunakan key pair yang sudah dibuat sebelumnya untuk mengakses server melalui SSH. Penggunaan key pair memastikan akses server tetap aman.

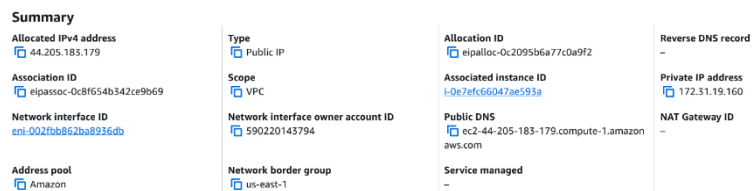
6. Memasukkan Jumlah Penyimpanan



Gambar 4. 22 Konfigurasi Storage Elasticsearch

Kapasitas penyimpanan yang digunakan lebih besar, yaitu 80 GB, karena Elasticsearch akan menyimpan data log dalam bentuk indeks. Semakin banyak data log yang masuk, maka kebutuhan penyimpanan juga akan semakin besar.

7. Mengaitkan dengan Elastic IP



Gambar 4. 23 Konfigurasi Elastic IP Elasticsearch

Elastic IP dialokasikan dan dihubungkan ke instance Elasticsearch agar alamat IP tidak berubah ketika server di-restart. Hal ini penting untuk memastikan koneksi dari Wazuh Manager tetap stabil.

8. Konfigurasi Security Group dan port

Security group rule ID	IP version	Type	Protocol	Port range	Source
sgr-06440d53f597dc12c	IPv4	SSH	TCP	22	0.0.0.0/0
sgr-0f6cd7730aa811926	IPv4	Custom TCP	TCP	9300	0.0.0.0/0
sgr-06f2ec95818d15a03	IPv4	Custom TCP	TCP	9200	0.0.0.0/0

Gambar 4. 24 Konfigurasi Security Group Elasticsearch

Pada tahap ini dilakukan konfigurasi Security Group untuk membuka port yang dibutuhkan, yaitu:

- e. Port 22: Digunakan untuk akses SSH (remote server).
- f. Port 9300: Digunakan untuk komunikasi internal antar node Elasticsearch (jika menggunakan cluster).

- g. Port 9200: Digunakan untuk akses layanan Elasticsearch melalui HTTP dan komunikasi dengan Wazuh Manager.

4.3.4 Instance Machine Learning

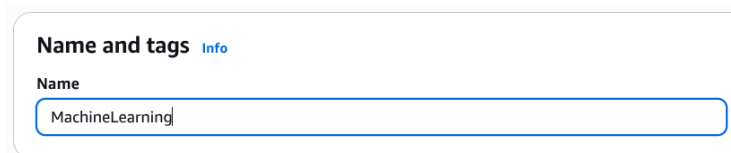
1. Launch Instance EC2



Gambar 4. 25 Launch Instance Machine Learning

Pembuatan instance dimulai dengan memilih menu Launch Instance pada layanan Amazon EC2 di Amazon Web Services. Instance ini digunakan untuk menjalankan proses pelatihan dan inferensi model Random Forest dalam mendeteksi anomali keamanan pada web server.

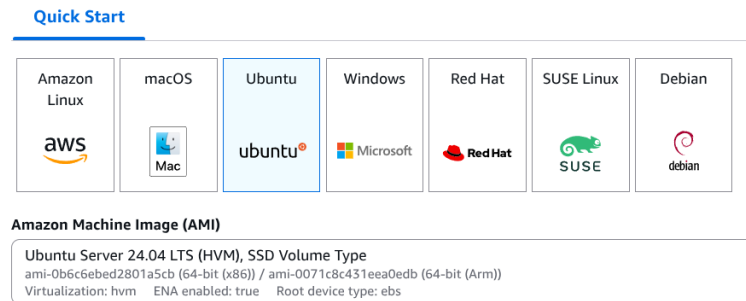
2. Memberi Name and Tags



Gambar 4. 26 Konfigurasi Name and Tags Machine Learning

Instance diberi nama “MachineLearningServer” agar mudah dikenali pada dashboard AWS. Penamaan ini bertujuan untuk membedakan server Machine Learning dengan server lainnya seperti Wazuh Manager, Wazuh Agent, dan Elasticsearch.

3. Memilih Amazon Machine Image (AMI)



Gambar 4. 27 Pemilihan AMI Machine Learning

Pada tahap ini dipilih Ubuntu Server 22.04 LTS sebagai sistem operasi. Ubuntu dipilih karena mendukung instalasi library Python dan framework machine learning seperti TensorFlow atau Scikit-learn yang digunakan dalam implementasi sistem deteksi phishing.

4. Memilih Tipe Instance



Gambar 4. 28 P emilihan Tipe Instance Machine Learning

Untuk server Machine Learning digunakan spesifikasi yang lebih tinggi, yaitu t3.medium atau setara (2 vCPU dan 4 GB RAM). Spesifikasi ini dipilih karena proses pelatihan (training) dan pengujian (testing) model machine learning membutuhkan sumber daya yang lebih besar dibandingkan server lainnya.

5. Membuat Key Pair

Create key pair ✕

Key pair name
Key pairs allow you to connect to your instance securely.

machineLearningEngine

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ **RSA**
RSA encrypted private and public key pair

☐ **ED25519**
ED25519 encrypted private and public key pair

Private key file format

☒ **.pem**
For use with OpenSSH

☐ **.ppk**
For use with PuTTY

Gambar 4. 29 Konfigurasi Key Pair Machine Learning

Pada tahap ini digunakan key pair yang sudah dibuat sebelumnya untuk mengakses server melalui SSH. Penggunaan key pair memastikan akses ke server tetap aman.

6. Memasukkan Jumlah Penyimpanan

▼ **Configure storage** [Info](#)

1x 30 GiB gp3 Root volume, 3000 IOPS, Not encrypted

Gambar 4. 30 Konfigurasi Storage Machine Learning

Kapasitas penyimpanan yang digunakan adalah sebesar 30 GB, yang digunakan untuk menyimpan dataset, file model machine learning, serta hasil proses analisis.

7. Mengaitkan dengan Elastic IP

Summary			
Allocated IPv4 address s2-71-135-122	Type Public IP	Allocation ID eipalloc-0869da680057c4080	Reverse DNS record -
Association ID eipassoc-046c4d22ab78d4941	Scope VPC	Associated instance ID i-0eafe6935bf77c592	Private IP address 172.31.22.249
Network interface ID eni-0fcbdac218597bec3	Network interface owner account ID 590220145794	Public DNS ec2-52-71-135-122.compute-1.amazonaws.com	NAT Gateway ID -
Address pool Amazon	Network border group us-east-1	Service managed -	

Gambar 4. 31 Konfigurasi Elastic IP Machine Learning

Elastic IP dialokasikan dan dihubungkan ke instance Machine Learning agar alamat IP tidak berubah saat server di-restart. Hal ini penting untuk menjaga kestabilan koneksi antara server Machine Learning dengan Wazuh Manager atau sistem lainnya.

8. Konfigurasi Security Group dan port

Security group rule ID	IP version	Type	Protocol	Port range	Source
sgr-012b628ab4e95a191	IPv4	SSH	TCP	22	0.0.0.0/0
sgr-06121d43b5669317b	IPv4	HTTPS	TCP	443	0.0.0.0/0
sgr-02f46c5b88b0c2469	IPv4	HTTP	TCP	80	0.0.0.0/0

Gambar 4. 32 Konfigurasi Security Group Machine Learning

Pada tahap ini dilakukan konfigurasi Security Group dengan membuka beberapa port yang dibutuhkan, yaitu:

- f. Port 22: Digunakan untuk akses SSH (remote server).
- g. Port 80: Digunakan untuk akses HTTP.
- h. Port 443: Digunakan untuk akses HTTPS yang lebih aman.

4.4 Implementasi Wazuh

4.4.1 Instalasi Wazuh Manager dan Wazuh Dashboard

Wazuh Dashboard digunakan pada tahap awal instalasi untuk verifikasi koneksi agent. Pada arsitektur final sistem monitoring dilakukan menggunakan Kibana yang terhubung langsung ke Elasticsearch.

1. Melakukan koneksi SSH.

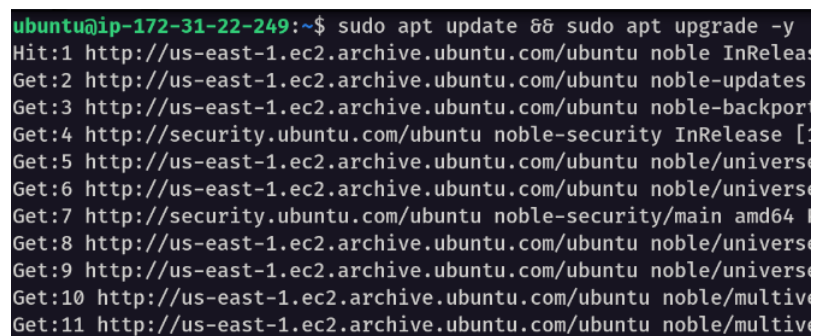
```
(root@mhdrazif)-[/home/mhdrazif/Documents/key pair]
# chmod 400 "wazuh-manager.pem"

(root@mhdrazif)-[/home/mhdrazif/Documents/key pair]
# ssh -i "wazuh-manager.pem" ubuntu@ec2-52-71-135-122.compute-1.amazonaws.com
The authenticity of host 'ec2-52-71-135-122.compute-1.amazonaws.com' can't be established.
ED25519 key fingerprint is: SHA256:LxlpiaV1wT4IFtzC0sejzGF01/sm
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint]) yes
Warning: Permanently added 'ec2-52-71-135-122.compute-1.amazonaws.com' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1018-aws x86_64)
```

Gambar 4. 33 Koneksi SSH Wazuh Manager

Berdasarkan Gambar 4.33, dilakukan koneksi ke server Wazuh Manager menggunakan SSH melalui terminal. Proses koneksi dilakukan dengan menggunakan file key pair (.pem) yang telah dibuat saat pembuatan instance. Koneksi SSH ini bertujuan untuk mengakses server secara remote agar dapat melakukan proses instalasi dan konfigurasi Wazuh.

2. Update dan Upgrade Sistem.



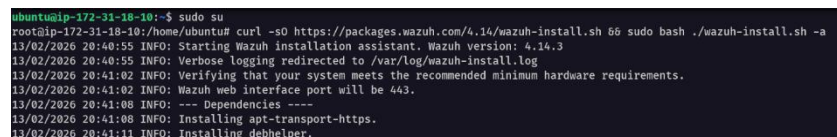
```
ubuntu@ip-172-31-22-249:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [10.9 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe InRelease
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe InRelease
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 InRelease
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe InRelease
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe InRelease
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse InRelease
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse InRelease
```

Gambar 4. 34 Update dan Upgrade Wazuh Manager

Tujuan dari melakukan perintah Sudo apt update dan Sudo apt upgrade adalah untuk memastikan semua paket sistem berada pada versi terbaru sehingga proses instalasi Wazuh dapat berjalan tanpa error.

3. Instalasi Wazuh Manager.

Pada tahap ini dilakukan instalasi Wazuh Manager menggunakan script instalasi resmi. Proses instalasi akan mengunduh dan mengkonfigurasi komponen yang dibutuhkan secara otomatis. Perintah yang digunakan yaitu `curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a`



```
ubuntu@ip-172-31-18-10:~$ sudo su
root@ip-172-31-18-10:/home/ubuntu# curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a
13/02/2026 20:40:55 INFO: Starting Wazuh installation assistant. Wazuh version: 4.14.3
13/02/2026 20:40:55 INFO: Verbose logging redirected to /var/log/wazuh-install.log
13/02/2026 20:41:02 INFO: Verifying that your system meets the recommended minimum hardware requirements.
13/02/2026 20:41:02 INFO: Wazuh web interface port will be 443.
13/02/2026 20:41:08 INFO: --- Dependencies ---
13/02/2026 20:41:08 INFO: Installing apt-transport-https.
13/02/2026 20:41:11 INFO: Installing debhelper.
```

Gambar 4. 35 Instalasi Wazuh Manager

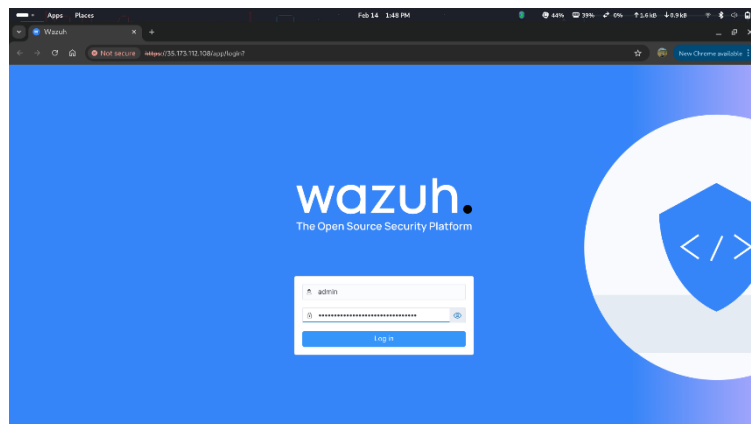
Jika proses instalasi berhasil, maka service Wazuh Manager akan berjalan pada server. Keberhasilan instalasi dapat dicek

dengan memastikan status service dalam keadaan active (running).

```
ubuntu@ip-172-31-18-10:~$ sudo systemctl status wazuh-manager.service
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-02-15 09:55:15 UTC; 2min 42s ago
     Process: 615 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 211 (limit: 4667)
   Memory: 785.6M (peak: 786.1M)
      CPU: 28.910s
   CGroup: /system.slice/wazuh-manager.service
```

4. Login Pada Wazuh Dashboard.

Setelah instalasi selesai, langkah selanjutnya adalah mengakses Wazuh Dashboard melalui browser menggunakan alamat IP publik (35.173.112.108) dari server.



Gambar 4. 36 Halaman Login Wazuh Dashboard

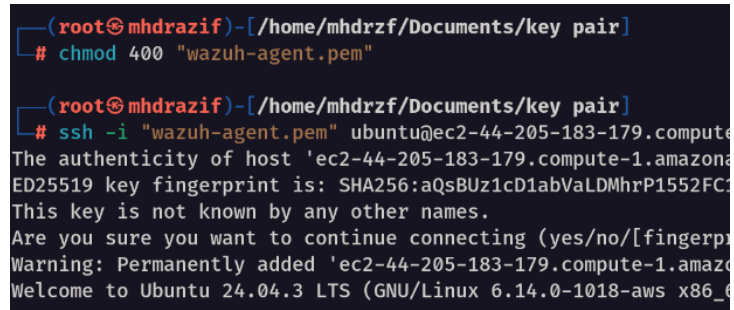
Pada halaman login kita dapat masuk ke Wazuh Dashboard menggunakan user dan password yang kita dapatkan disaat melakukan penginstallan wazuh, seperti pada gambar 4. 37.

```
13/02/2026 20:48:33 INFO: You can access the web in
User: admin
Password: EWe4mqileTgUsue9k+XOX.t.nNSlyyUk
13/02/2026 20:48:33 INFO: Installation finished.
root@ip-172-31-18-10:/home/ubuntu#
```

Gambar 4. 37 User dan Password Wazuh Manager

4.4.2 Instalasi dan Konfigurasi Wazuh Agent

1. Koneksi SSH.



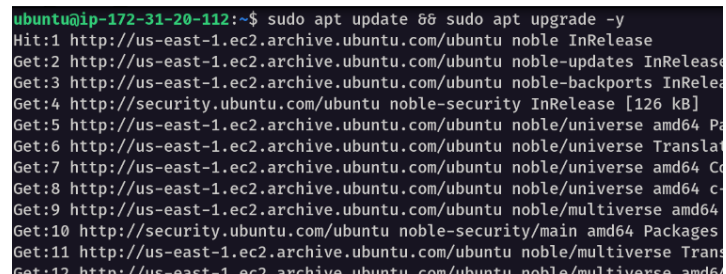
```
(root@mhdrazif)-[/home/mhdrazif/Documents/key pair]
# chmod 400 "wazuh-agent.pem"

(root@mhdrazif)-[/home/mhdrazif/Documents/key pair]
# ssh -i "wazuh-agent.pem" ubuntu@ec2-44-205-183-179.compute-1.amazonaws.com
The authenticity of host 'ec2-44-205-183-179.compute-1.amazonaws.com' can't be
established.
ED25519 key fingerprint is: SHA256:aQsBUz1cD1abVaLDMhrP1552FC1
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint]) yes
Warning: Permanently added 'ec2-44-205-183-179.compute-1.amazonaws.com'
to the list of known hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1018-aws x86_64)
```

Gambar 4. 38 Koneksi SSH Menggunakan Wazuh Agent

Berdasarkan Gambar 4.38, dilakukan koneksi SSH ke server Wazuh Agent menggunakan key pair (.pem) melalui terminal. Proses ini dilakukan agar server client dapat diakses secara remote untuk melakukan instalasi dan konfigurasi Wazuh Agent.

2. Update dan Upgrade Sistem.



```
ubuntu@ip-172-31-20-112:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [12.6 MB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe Translation [12.6 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 C-Files [12.6 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 c-Files [12.6 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [12.6 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [12.6 kB]
Get:11 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse Translation [12.6 kB]
Get:12 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/multiverse amd64 C-Files [12.6 kB]
```

Gambar 4. 39 Update dan Upgrade Wazuh Agent

Pada tahap ini dilakukan pembaruan sistem dengan menjalankan perintah update dan upgrade. Langkah ini bertujuan untuk memastikan semua paket sistem berada pada versi terbaru dan menghindari error saat proses instalasi.

3. Instalasi dan Konfigurasi Wazuh Agent.

```

ubuntu@ip-172-31-20-112:~$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.3-1_amd64.deb && sudo WAZUH_MA
NAGER="35.173.112.108" WAZUH_AGENT_NAME="wazuh-agent-1" dpkg -i ./wazuh-agent_4.14.3-1_amd64.deb
--2026-02-16 16:58:30-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.3-1_amd64.deb
Resolving packages.wazuh.com (packages.wazuh.com)... 13.226.209.93, 13.226.209.39, 13.226.209.78, ...
Connecting to packages.wazuh.com (packages.wazuh.com)|13.226.209.93|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13213254 (13M) [application/vnd.debian.binary-package]
Saving to: 'wazuh-agent_4.14.3-1_amd64.deb'

wazuh-agent_4.14.3-1_amd64.deb 100%[=====] 12.68M --.-KB/s in 0.04s

2026-02-16 16:58:30 (352 MB/s) - 'wazuh-agent_4.14.3-1_amd64.deb' saved [13213254/13213254]

Selecting previously unselected package wazuh-agent.
(Reading database ... 71753 files and directories currently installed.)
Preparing to unpack .../wazuh-agent_4.14.3-1_amd64.deb ...
Unpacking wazuh-agent (4.14.3-1) ...
Setting up wazuh-agent (4.14.3-1) ...
ubuntu@ip-172-31-20-112:~$ sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-agent.service → /usr/lib/systemd/system/wazuh-agent.service.
ubuntu@ip-172-31-20-112:~$

```

Gambar 4. 40 Instalasi Wazuh Agent

Pada tahap ini dilakukan instalasi Wazuh Agent pada server client. Proses instalasi dilakukan dengan menambahkan repository Wazuh dan menginstal paket wazuh-agent.

```

root@ip-172-31-19-160:/home/ubuntu# systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-02-13 21:20:26 UTC; 5min ago
     Process: 11044 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, sta
     Process: 11771 ExecReload=/usr/bin/env /var/ossec/bin/wazuh-control reload (code=exited, s
       Tasks: 32 (limit: 1017)
      Memory: 249.1M (peak: 258.2M)
         CPU: 15.881s
        CGroup: /system.slice/wazuh-agent.service

```

Gambar 4. 41 Status Wazuh Agent

Setelah konfigurasi selesai, service Wazuh Agent dijalankan dan dicek statusnya. Jika status menunjukkan active (running), maka agent berhasil terinstal dan siap mengirimkan data log ke Wazuh Manager.

4.5 Integrasi Penyimpanan Log ke Elasticsearch

Pada tahap ini dilakukan perubahan arsitektur penyimpanan log dari sistem bawaan Wazuh Manager menuju sistem penyimpanan terpusat berbasis Elasticsearch. Perubahan ini bertujuan untuk meningkatkan fleksibilitas pengolahan data, mendukung integrasi Machine Learning, serta mempermudah proses indexing dan query log dalam jumlah besar. Secara default, Wazuh Manager menyimpan alert dalam format JSON pada direktori lokal:

```

ubuntu@ip-172-31-18-10:~$ sudo tail -n 1 /var/ossec/logs/alerts/alerts.json | python3 -m json.tool | head -n 80
{
  "timestamp": "2026-02-22T22:39:18.952+0000",
  "rule": {
    "level": 3,
    "description": "Successful sudo to ROOT executed.",
    "id": "5402",
    "mitre": {
      "id": [
        "T1548.003"
      ],
      "tactic": [
        "Privilege Escalation",
        "Defense Evasion"
      ],
      "technique": [
        "Sudo and Sudo Caching"
      ]
    },
    "firedtimes": 1,
    "mail": false,
    "groups": [
      "syslog",
      "sudo"
    ],
    "pci_dss": [
      "10.2.5",
      "10.2.2"
    ],
    "gpg13": [
      "7.6",
      "7.8",
      "7.9"
    ]
  }
}

```

Gambar 4. 42 Alert Dalam Format JSON

Penyimpanan lokal ini bersifat file-based dan tidak mendukung pencarian terstruktur secara efisien dalam skala data besar. Oleh karena itu, dilakukan integrasi dengan Elasticsearch sebagai sistem penyimpanan berbasis search engine.

4.5.1 Arsitektur Pengiriman Log

Keuntungan Integrasi Elasticsearch dalam Sistem. Alur prosesnya adalah sebagai berikut:

1. Wazuh Manager menghasilkan alert dalam format JSON.
2. Filebeat membaca file `alerts.json`.
3. Filebeat mengirim data ke Elasticsearch melalui REST API.
4. Elasticsearch menerima dan menyimpan data dalam bentuk index.

Skema alus data Wazuh Manager: `alerts.json` → Filebeat → Elasticsearch (index `wazuh-alerts-*`)

Dengan pendekatan ini, log tidak lagi hanya tersimpan sebagai file statis, tetapi menjadi data terindeks yang dapat di-query secara cepat dan fleksibel.

4.5.2 Struktur Data Log dalam Elasticsearch

Data yang tersimpan dalam Elasticsearch berbentuk dokumen JSON dengan struktur terorganisir. Beberapa field penting yang digunakan dalam penelitian ini antara lain:

1. `full_log` → Isi lengkap log mentah
2. `rule.level` → Tingkat severity rule
3. `rule.description` → Deskripsi jenis alert
4. `rule.firedtimes` → Jumlah kemunculan alert
5. `@timestamp` → Waktu kejadian
6. `agent.name` → Nama host sumber log

Struktur ini sangat penting karena:

1. Mendukung proses filtering di Kibana
2. Mempermudah ekstraksi dataset untuk Machine Learning
3. Memungkinkan analisis berbasis waktu (time-series analysis)

Dengan adanya field terstruktur, proses feature engineering dapat dilakukan secara sistematis tanpa perlu parsing manual log mentah.

4.5.3 Keuntungan Integrasi Elasticsearch dalam Sistem

Integrasi Elasticsearch memberikan beberapa peningkatan signifikan dibanding penyimpanan default berbasis file:

1. Scalability: Elasticsearch mampu menangani log dalam jumlah besar dengan performa stabil.
2. Fast Querying: Log dapat dicari berdasarkan timestamp, rule level, atau agent secara cepat.
3. Structured Data: Data berbentuk JSON terstruktur sehingga mudah diproses Machine Learning.
4. Time-based Analysis: Mendukung analisis pola serangan berbasis waktu.

5. Interoperability: Data dapat digunakan oleh berbagai sistem, termasuk Kibana dan Machine Learning Engine.

4.6 Implementasi Kibana sebagai Dashboard Monitoring

Setelah log dipusatkan di Elasticsearch, diperlukan antarmuka visualisasi untuk melakukan monitoring. Pada tahap awal implementasi, Wazuh Dashboard digunakan untuk verifikasi koneksi agent dan pembentukan alert. Namun pada arsitektur final, monitoring dilakukan menggunakan Kibana yang terhubung langsung ke Elasticsearch.

1. Konfigurasi Kibana:

```
server.host: "0.0.0.0"  
server.port: 5601  
  
elasticsearch.hosts: ["http://localhost:9200"]
```

Gambar 4. 43 Konfigurasi Kibana

File konfigurasi kibana.yml digunakan untuk mengatur koneksi antara Kibana dan Elasticsearch. Parameter utama yang dikonfigurasi adalah `elasticsearch.hosts`, yang mengarah ke alamat Elasticsearch tempat data alert Wazuh disimpan. Selain itu, parameter `server.host` dan `server.port` digunakan untuk menentukan alamat dan port akses dashboard Kibana. Pada penelitian ini, Kibana digunakan untuk memvisualisasikan index `wazuh-alerts-*` serta memverifikasi bahwa data log berhasil tersimpan di Elasticsearch sebelum diproses oleh sistem Machine Learning.

4.7 Implementasi dan Integrasi Machine Learning

Pada tahap ini dilakukan pengembangan sistem dengan menambahkan Machine Learning sebagai lapisan analisis tambahan di atas sistem SIEM berbasis Wazuh. Jika Wazuh berfungsi sebagai sistem deteksi berbasis aturan (rule-based detection), maka Machine Learning berfungsi sebagai mekanisme klasifikasi berbasis data (data-driven classification) yang mempelajari pola historis log keamanan.

Integrasi Machine Learning dilakukan menggunakan algoritma Random Forest dengan pendekatan supervised learning sebagaimana dijelaskan pada BAB II. Model dilatih menggunakan data log yang telah diberi label (normal dan anomali), kemudian diimplementasikan dalam sistem deteksi secara realtime untuk membantu memvalidasi alert yang dihasilkan oleh Wazuh.

Implementasi Machine Learning dibagi menjadi beberapa tahapan, yaitu:

1. Pengambilan dataset dari Elasticsearch
2. Preprocessing dan feature engineering
3. Pelatihan model
4. Evaluasi model
5. Integrasi deteksi realtime dan notifikasi Telegram

4.7.1 Arsitektur Modular Machine Learning

Sistem Machine Learning dibangun secara modular, dimana setiap proses dipisahkan dalam file Python yang berbeda, yaitu:

1. `01_ambil_data.py` : Pengambilan dataset
2. `02_preprocessing.py` : Pembersihan dan pembuatan fitur
3. `03_latih_model.py` : Pelatihan model
4. `04_evaluasi_model.py` : Evaluasi model
5. `05_deteksi_realtime_telegram.py` : Deteksi realtime

Konfigurasi umum seperti alamat Elasticsearch, nama index, serta nama file model dipusatkan dalam file `konfigurasi.py`, yang berisi:

```
# konfigurasi.py
# Konfigurasi dasar untuk semua script

ELASTIC_BASE_URL = "http://100.26.168.156:9200"
INDEX_PATTERN = "wazuh-alerts-*"

# File output
FILE_RAW = "wazuh_logs_raw.csv"
FILE_CLEAN = "wazuh_logs_clean.csv"

# Model & laporan
FILE_MODEL = "model_random_forest.pkl"
FILE_REPORT = "classification_report.txt"
FILE_CONFUSION = "confusion_matrix.csv"

# Parameter pengambilan data
JUMLAH_DATA = 5000
```

Gambar 4. 46 Script konfigurasi.py

4.7.2 Pengambilan Dataset dari Elasticsearch

```
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# nano 01 ambil_data.py
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# python 01 ambil_data.py
[INFO] Mengambil data dari: http://100.26.168.156:9200/wazuh-alerts-*/_search
[INFO] Total dokumen diterima: 4993
[INFO] Data mentah disimpan: wazuh_logs_raw.csv
[INFO] Contoh 5 baris pertama:
rule_level rule_id ... full_log location
0 5 5710 ... Feb 23 16:39:19 ip-172-31-20-112 sshd[7105]: F... journald
1 5 5718 ... Feb 23 16:39:17 ip-172-31-20-112 sshd[7105]: U... journald
2 5 5503 ... Feb 23 16:39:17 ip-172-31-20-112 sshd[7105]: p... journald
3 5 5710 ... Feb 23 16:38:47 ip-172-31-20-112 sshd[7103]: F... journald
4 5 5503 ... Feb 23 16:38:45 ip-172-31-20-112 sshd[7103]: p... journald

[5 rows x 10 columns]
[INFO] Total baris: 4993
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# ls
01_ambil_data.py __pycache__ konfigurasi.py wazuh_logs_raw.csv
```

Gambar 4. 47 Script Pengambilan Data

Dataset diambil langsung dari index `wazuh-alerts-*` pada Elasticsearch menggunakan REST API dengan endpoint: `http://IP_ELASTIC:9200/wazuh-alerts-*/_search`

Field yang digunakan sebagai dasar pembentukan dataset antara lain:

1. rule.level
2. rule.description
3. rule.firedtimes
4. @timestamp

5. agent.name

6. full_log

Data diambil berdasarkan jumlah tertentu (misalnya 5000 dokumen terbaru) dan disimpan dalam format CSV sebagai dataset mentah. Penggunaan Elasticsearch sebagai sumber dataset memastikan bahwa data yang digunakan untuk Machine Learning identik dengan data yang digunakan untuk monitoring di Kibana, sehingga konsistensi sistem tetap terjaga.

4.7.3 Preprocessing dan Feature Engineering

```
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# nano 02_preprocessing.py
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# python 02_preprocessing.py
[INFO] Membaca dataset mentah: wazuh_logs_raw.csv
[INFO] Total baris: 4993
[INFO] Dataset bersih disimpan: wazuh_logs_clean.csv
[INFO] Distribusi label:
label
1      1502
0       723
Name: count, dtype: int64
[INFO] Contoh dataset clean (5 baris):
   firedtimes  rule_level  hour  is_night  is_ssh  log_length  label
0          23           5    16         0       1         118      1
1          19           5    16         0       1         119      1
2          21           5    16         0       1         158      1
3          22           5    16         0       1         115      1
4          20           5    16         0       1         155      1
[INFO] Total baris clean: 2225
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# _
```

Gambar 4. 48 Preprocessing

Tahap preprocessing dilakukan untuk membersihkan dan mengubah data mentah menjadi fitur numerik yang dapat digunakan oleh algoritma Machine Learning.

Langkah-langkah preprocessing meliputi:

1. Menghapus data dengan nilai kosong pada kolom penting
2. Mengubah timestamp menjadi format datetime
3. Mengekstrak jam kejadian
4. Membuat fitur tambahan berdasarkan karakteristik log

Tabel 4. 1 Daftar Fitur Machine Learning

Fitur	Penjelasan
firedtimes	Jumlah kemunculan alert

rule_level	Tingkat severity rule
hour	Jam kejadian
is_night	Indikator kejadian pada jam 00.00–05.00
is_ssh	Indikator aktivitas SSH
log_length	Panjang karakter log

Labeling dilakukan dengan aturan:

1. Jika fireddtimes > 10 ATAU rule_level $\geq 7 \rightarrow$ label = 1 (anomali)
2. label = 0 (normal)

Proses ini merupakan implementasi pendekatan supervised learning sebagaimana dijelaskan pada BAB II. Dengan adanya label kelas, model dapat mempelajari hubungan antara fitur numerik dan kategori aktivitas keamanan. Labeling ini dilakukan dalam lingkungan simulasi terkontrol untuk membangun dataset pelatihan yang terstruktur.

4.7.4 Pelatihan Model Random Forest

```
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# nano 03_latih_model.py
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# python 03_latih_model.py
[INFO] Membaca dataset clean: wazuh_logs_clean.csv
[INFO] Total baris: 2225
[INFO] Data training: 1557
[INFO] Data testing : 668
[INFO] Melatih model Random Forest...
✓ Model berhasil disimpan: model_random_forest.pkl
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# ls
01_ambil_data.py      03_latih_model.py  konfigurasi.py      wazuh_logs_clean.csv
02_preprocessing.py  __pycache__       model_random_forest.pkl  wazuh_logs_raw.csv
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml#
```

Gambar 4. 49 Pelatihan Model Random Forest

Setelah proses preprocessing dan pembuatan fitur selesai, tahap selanjutnya adalah pelatihan model Machine Learning menggunakan algoritma Random Forest Classifier. Random Forest dipilih karena memiliki beberapa keunggulan dalam menangani data keamanan, yaitu:

1. Mampu menangani data numerik dengan baik
2. Tidak mudah overfitting karena menggunakan banyak pohon keputusan
3. Stabil terhadap data yang tidak seimbang
4. Memiliki performa yang baik pada klasifikasi berbasis pola log

a. Pembagian Dataset

Dataset yang telah dibersihkan (wazuh_logs_clean.csv) kemudian dipisahkan menjadi dua bagian:

- 1) 70% data training
- 2) 30% data testing

Pembagian ini dilakukan menggunakan fungsi `train_test_split` dari library Scikit-Learn dengan parameter:

- 1) `test_size = 0.3`
- 2) `random_state = 42`
- 3) `stratify = y`

Penggunaan parameter `stratify` bertujuan untuk menjaga proporsi distribusi label (normal dan anomali) agar tetap seimbang pada data training dan testing. Sebagai contoh, dari total 2225 data bersih, diperoleh:

- 1) Data training: 1557 baris
- 2) Data testing: 668 baris

b. Proses Pelatihan Model

Model dibuat menggunakan:

```
RandomForestClassifier(
    n_estimators=300,
    random_state=42,
    class_weight="balanced"
)
```

Gambar 4. 50 Script Pelatihan Model

Penjelasan parameter:

- 1) `n_estimators = 300` Artinya model membangun 300 pohon keputusan. Semakin banyak pohon, semakin stabil hasil prediksi.
- 2) `random_state = 42` Digunakan agar hasil pelatihan konsisten setiap kali script dijalankan.
- 3) `class_weight = "balanced"` Digunakan untuk menangani kemungkinan ketidakseimbangan jumlah data antara kelas normal dan anomali.

Selama proses training, model akan:

- 1) Membuat banyak pohon keputusan berdasarkan subset data acak.
- 2) Setiap pohon melakukan klasifikasi berdasarkan fitur seperti:
 - a) `firedtimes`
 - b) `rule_level`
 - c) `hour`
 - d) `is_night`
 - e) `is_ssh`
 - f) `log_length`
- 3) Hasil akhir ditentukan berdasarkan voting mayoritas dari seluruh pohon.

c. Penyimpanan Model

Setelah proses pelatihan selesai, model disimpan dalam bentuk file: `model_random_forest.pkl`. Tujuan penyimpanan model adalah:

- 1) Menghindari proses training ulang setiap kali sistem dijalankan
- 2) Mempercepat proses deteksi realtime
- 3) Memastikan model yang digunakan pada tahap evaluasi dan realtime sama dengan model yang dilatih

d. Hubungan Model dengan Sistem Realtime

Model yang telah dilatih tidak hanya digunakan untuk evaluasi, tetapi juga diintegrasikan ke dalam sistem deteksi realtime. Saat sistem mengambil alert terbaru dari Elasticsearch, fitur diekstrak menggunakan skema yang sama seperti saat training, kemudian diberikan ke model untuk diprediksi. Konsistensi fitur antara tahap training dan tahap realtime sangat penting agar hasil prediksi akurat.

4.7.5 Evaluasi Model

```
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# nano 04_evaluasi_model.py
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# python3 04_evaluasi_model.py
[INFO] Membaca dataset clean: wazuh_logs_clean.csv
[INFO] Total baris: 2225
[INFO] Model dimuat: model_random_forest.pkl

=== Classification Report ===
              precision    recall  f1-score   support

     0       1.0000      1.0000      1.0000        217
     1       1.0000      1.0000      1.0000        451

 accuracy          1.0000
 macro avg          1.0000
weighted avg          1.0000

=== Confusion Matrix ===
[[217  0]
 [ 0 451]]
✓ Report disimpan: classification_report.txt
✓ Confusion matrix disimpan: confusion_matrix.csv
```

Gambar 4. 51 Evaluasi Random Forest

Setelah model Random Forest selesai dilatih, tahap selanjutnya adalah melakukan evaluasi untuk mengukur performa model dalam mengklasifikasikan data menjadi dua kelas, yaitu normal (0) dan anomali (1). Evaluasi dilakukan menggunakan data testing yang sebelumnya telah dipisahkan sebesar 30% dari total dataset.

1. Metode Evaluasi yang Digunakan

Metrik evaluasi yang digunakan dalam penelitian ini adalah:

- a. Accuracy: Mengukur persentase total prediksi yang benar terhadap seluruh data testing.
- b. Precision: Mengukur seberapa banyak prediksi anomali yang benar-benar anomali.

- c. Recall: Mengukur kemampuan model dalam mendeteksi seluruh data anomali yang ada.
- d. F1-Score: Rata-rata harmonik antara precision dan recall.
- e. Confusion Matrix: Digunakan untuk melihat distribusi prediksi benar dan salah secara detail.

2. Hasil Classification Report

Berdasarkan hasil evaluasi menggunakan `classification_report`, diperoleh hasil sebagai berikut:

- 1) Precision: 1.000
- 2) Recall: 1.000
- 3) F1-Score: 1.000
- 4) Accuracy: 1.000

Walaupun model menunjukkan nilai Accuracy, Precision, Recall, dan F1-score sebesar 1.000, hasil ini diperoleh pada dataset hasil simulasi dalam lingkungan terkontrol. Proses labeling dilakukan berdasarkan aturan tertentu, seperti threshold pada `rule_level` dan `firedtimes`, sehingga pola antara kelas normal dan anomali relatif jelas terpisah. Kondisi ini memungkinkan model Random Forest mengklasifikasikan data testing tanpa kesalahan.

Namun demikian, pada implementasi di lingkungan produksi dengan data log yang lebih kompleks dan beragam, performa model berpotensi berbeda. Oleh karena itu, diperlukan pengujian lanjutan menggunakan dataset yang lebih besar dan lebih variatif untuk memastikan generalisasi model.

3. Analisis Confusion Matrix

Confusion Matrix yang dihasilkan berbentuk:

```
[[ 217      0]
 [   0    451]]
```

Interpretasi:

- 217 data normal diprediksi sebagai normal (True Negative)
- 451 data anomali diprediksi sebagai anomali (True Positive)
- 0 False Positive
- 0 False Negative

Model tidak melakukan kesalahan klasifikasi pada data testing, seluruh anomali berhasil terdeteksi dan tidak ada aktivitas normal yang salah diklasifikasikan sebagai anomali.

4.7.6 Implementasi Deteksi Realtime dan Notifikasi Telegram

```
=====
[BUKTI] Alert diambil dari Elasticsearch
ID : H-6bi5wBsqc_ePbgXA8d
Time : 2026-02-23T17:45:36.752Z
Agent: wazuh-agent-1
Rule : syslog: User missed the password more than one time
[BUKTI] Fitur diproses (input ML):
  firedtimes rule_level hour is_night is_ssh log_length
           2         10   17         0     1         148
[BUKTI] Prediksi ML: 1 (1=anomali, 0=normal)
=====
[INFO] Telegram status: 200
✅ Notifikasi Telegram terkirim.

=====
[BUKTI] Alert diambil dari Elasticsearch
ID : Hu6bi5wBsqc_ePbgXA8d
Time : 2026-02-23T17:45:34.751Z
Agent: wazuh-agent-1
Rule : sshd: Attempt to login using a non-existent user
[BUKTI] Fitur diproses (input ML):
  firedtimes rule_level hour is_night is_ssh log_length
           6         5   17         0     1         117
[BUKTI] Prediksi ML: 0 (1=anomali, 0=normal)
=====
```

Gambar 4. 52 Deteksi Realtime

Setelah model Random Forest selesai dilatih dan dievaluasi, tahap selanjutnya adalah mengintegrasikan model tersebut ke dalam sistem deteksi secara realtime. Pada tahap ini, sistem tidak hanya melakukan analisis secara offline menggunakan dataset statis, tetapi langsung memproses alert terbaru yang masuk ke Elasticsearch.

Script yang digunakan pada tahap ini adalah: 05_deteksi_realtime_telegram.py. Script ini berfungsi sebagai penghubung antara Elasticsearch, model Machine Learning, dan Telegram sebagai media notifikasi.

1. Alur Kerja Sistem Realtime

Alur kerja sistem realtime adalah sebagai berikut:

- a. Script mengambil alert terbaru dari Elasticsearch.
- b. Alert diekstrak menjadi fitur numerik.
- c. Fitur diproses oleh model Random Forest.
- d. Model menghasilkan prediksi (normal atau anomali).
- e. Jika hasil prediksi adalah anomali, sistem mengirim notifikasi Telegram.

2. Pengambilan Alert dari Elasticsearch

Script mengambil data menggunakan REST API dengan endpoint: http://IP ELASTIC:9200/wazuh-alerts-*/_search

Query dikonfigurasi untuk:

- a. Mengambil 5 alert terbaru
- b. Mengurutkan berdasarkan @timestamp secara descending
- c. Memfilter alert dengan `rule.level ≥ 3`

Filter ini bertujuan untuk menghindari pemrosesan event dengan tingkat severity rendah sehingga sistem lebih fokus pada alert yang relevan. Proses ini dijalankan setiap 20 detik menggunakan perulangan otomatis, sehingga sistem bekerja secara near realtime.

3. Ekstraksi Fitur untuk Prediksi

Setiap alert yang diambil dari Elasticsearch memiliki struktur JSON. Dari struktur tersebut, diekstrak fitur yang sama seperti pada tahap pelatihan model, yaitu:

- a. `firedtimes`
- b. `rule_level`
- c. `hour`
- d. `is_night`
- e. `is_ssh`

f. `log_length`

Konsistensi fitur antara tahap training dan tahap realtime sangat penting agar model dapat melakukan prediksi secara akurat.

4. Proses Prediksi Menggunakan Model

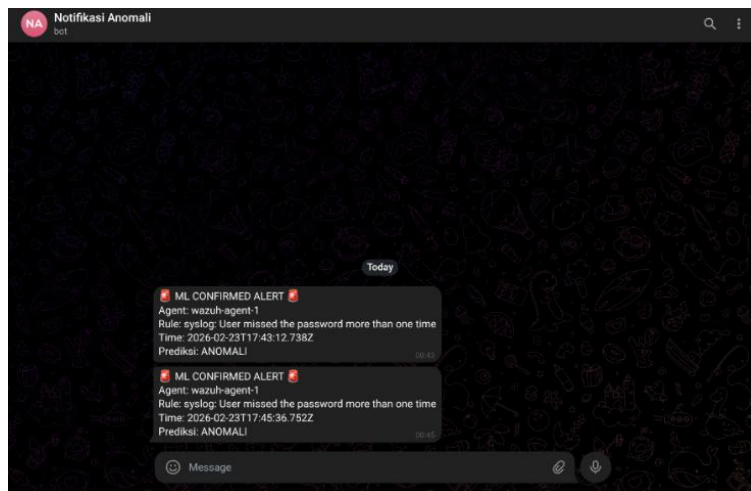
Model yang telah dilatih sebelumnya dimuat dari file: `model_random_forest.pkl`.

Setelah fitur disiapkan, model melakukan klasifikasi:

- a. 0 = Normal
- b. 1 = Anomali

Jika hasil prediksi adalah 1 (anomali), maka sistem akan melanjutkan ke tahap pengiriman notifikasi Telegram.

5. Integrasi Notifikasi Telegram



Gambar 4. 53 Notifikasi Telegram

Model memprediksi suatu alert sebagai anomali, sistem akan mengirim pesan melalui Telegram Bot API dengan parameter yang digunakan yaitu `TELEGRAM_TOKEN` dan `TELEGRAM_CHAT_ID`.

Format pesan yang dikirim adalah:

🚨 ML CONFIRMED ALERT 🚨

Agent: <nama_agent>

Rule: <deskripsi_rule>

Time: <timestamp>

Prediksi: ANOMALI

Penambahan teks “ML CONFIRMED ALERT” bertujuan untuk menegaskan bahwa notifikasi tersebut telah melalui proses konfirmasi Machine Learning, bukan sekadar alert mentah dari sistem.

4.8 Perbandingan Sistem Wazuh Tanpa Machine Learning dan Wazuh dengan Machine Learning

4.8.1 Perbandingan Arsitektur Sistem

Pada sistem tanpa Machine Learning, alur deteksi adalah:.

1. Web Server → Wazuh Agent → Wazuh Manager → Elasticsearch → Kibana

Sedangkan pada sistem dengan Machine Learning, arsitektur berkembang menjadi:

2. Web Server → Wazuh Agent → Wazuh Manager → Elasticsearch → Machine Learning → Telegram

Pada arsitektur ini, Machine Learning bertindak sebagai lapisan klasifikasi ulang sebelum notifikasi dikirim. Perubahan arsitektur ini menunjukkan bahwa ML tidak menggantikan Wazuh, tetapi menambahkan lapisan analisis tambahan di atas sistem rule-based.

4.8.2 Perbandingan Mekanisme Deteksi

Perbandingan mekanisme deteksi antara sistem tanpa Machine Learning dan sistem dengan Machine Learning dapat dijelaskan sebagai berikut:

1. Sistem Tanpa Machine Learning

Mekanisme deteksi pada sistem ini sepenuhnya berbasis rule Wazuh. Prosesnya meliputi:

- a. Log dikirim dari Wazuh Agent ke Wazuh Manager.
- b. Wazuh melakukan proses decoding dan rule matching.
- c. Jika rule terpenuhi, alert dihasilkan.
- d. Alert disimpan di Elasticsearch dan ditampilkan di Kibana.

Karakter utama sistem tanpa ML:

- a. Deteksi bergantung pada rule statis.
- b. Tidak terdapat proses klasifikasi ulang.
- c. Tidak tersedia evaluasi performa klasifikasi.
- d. Seluruh alert yang sesuai rule dianggap valid.
- e. Administrator melakukan analisis lanjutan secara manual.

Keterbatasan utama:

- a. Tidak mempertimbangkan pola historis data.
- b. Tidak memiliki mekanisme pembelajaran.
- c. Bergantung sepenuhnya pada rule yang telah didefinisikan.

2. Sistem Dengan Machine Learning

Pada sistem ini, mekanisme deteksi terdiri dari dua tahap, yaitu rule-based detection dan klasifikasi berbasis data.

Proses deteksi meliputi:

- a. Wazuh menghasilkan alert berdasarkan rule.
- b. Alert disimpan dalam Elasticsearch.
- c. Machine Learning mengambil alert terbaru.
- d. Fitur diekstrak dari alert.
- e. Model Random Forest melakukan klasifikasi.
- f. Hanya alert yang diprediksi sebagai anomali yang dikirim sebagai notifikasi.

Karakteristik utama sistem dengan ML:

- a. Menggunakan pendekatan rule-based + supervised learning.
- b. Memiliki proses klasifikasi ulang sebelum notifikasi.
- c. Tersedia evaluasi model menggunakan confusion matrix.

- d. Mempertimbangkan kombinasi fitur numerik.
- e. Berpotensi mengurangi beban analisis manual.

3. Perbedaan Inti Mekanisme

Perbedaan utama kedua sistem dapat dirangkum sebagai berikut:

Tabel 4. 2 Perbandingan Sistem Tanpa ML vs Dengan ML

Aspek	Tanpa ML	Dengan ML
Sumber keputusan	Rule statis	Rule + Model
Proses validasi ulang	Tidak ada	Ada
Evaluasi statistik	Tidak tersedia	Tersedia
Kemampuan belajar	Tidak ada	Ada (berdasarkan dataset)
Tingkat analisis	Single-layer	Multi-layer

Dengan demikian, sistem dengan Machine Learning memiliki mekanisme deteksi dua lapis (two-layer detection), sedangkan sistem tanpa Machine Learning hanya memiliki satu lapis deteksi berbasis rule.

4.8.3 Perbandingan Kuantitatif

Perbandingan kuantitatif dilakukan untuk mengukur performa sistem setelah integrasi Machine Learning. Evaluasi dilakukan menggunakan data testing sebesar 30% dari total dataset yang telah melalui proses preprocessing dan labeling.

Pada sistem tanpa Machine Learning, tidak dilakukan evaluasi berbasis confusion matrix karena sistem hanya menghasilkan alert berdasarkan rule yang terpicu. Oleh karena itu, tidak tersedia metrik klasifikasi seperti accuracy, precision, recall, maupun F1-score.

Sebaliknya, pada sistem dengan Machine Learning dilakukan evaluasi menggunakan confusion matrix dengan hasil sebagai berikut:

```

(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# nano 04_evaluasi_model.py
(ml-env) root@ip-172-31-31-58:/home/ubuntu/sistem_ml# python3 04_evaluasi_model.py
[INFO] Membaca dataset clean: wazuh_logs_clean.csv
[INFO] Total baris: 2225
[INFO] Model dimuat: model_random_forest.pkl

=== Classification Report ===
              precision    recall  f1-score   support

     0       1.0000      1.0000      1.0000       217
     1       1.0000      1.0000      1.0000       451

 accuracy          1.0000      1.0000      1.0000       668
 macro avg          1.0000      1.0000      1.0000       668
weighted avg          1.0000      1.0000      1.0000       668

=== Confusion Matrix ===
[[217  0]
 [ 0 451]]
✓ Report disimpan: classification_report.txt
✓ Confusion matrix disimpan: confusion_matrix.csv

```

Gambar 4. 54 Confusion Matrix

Matriks tersebut menunjukkan bahwa:

1. Sebanyak 217 data normal berhasil diklasifikasikan sebagai normal (True Negative).
2. Sebanyak 451 data anomali berhasil diklasifikasikan sebagai anomali (True Positive).
3. Tidak terdapat data normal yang salah diklasifikasikan sebagai anomali (False Positive = 0).
4. Tidak terdapat data anomali yang gagal terdeteksi (False Negative = 0).

Perhitungan Matriks:

1. Accuracy

Accuracy mengukur tingkat ketepatan prediksi secara keseluruhan dan dihitung menggunakan rumus:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Accuracy = \frac{451 + 217}{451 + 217 + 0 + 0}$$

$$Accuracy = \frac{668}{668}$$

$$Accuracy = 1.000 (100\%)$$

Nilai accuracy sebesar 100% menunjukkan bahwa seluruh data testing berhasil diklasifikasikan dengan benar pada dataset

simulasi yang digunakan.

2. Precision

Precision mengukur proporsi prediksi anomali yang benar-benar merupakan anomali. Rumus precision adalah:

$$Precision = \frac{TP}{TP + FP}$$

$$Precision = \frac{451}{451 + 0}$$

$$Precision = \frac{451}{451}$$

$$Precision = 1.000 (100\%)$$

Nilai precision 100% menunjukkan bahwa tidak terdapat kesalahan dalam memprediksi aktivitas normal sebagai anomali.

3. Recall

Recall mengukur kemampuan model dalam mendeteksi seluruh data anomali. Rumus recall adalah:

$$Recall = \frac{TP}{TP + FN}$$

$$Recall = \frac{451}{451 + 0}$$

$$Recall = \frac{451}{451}$$

$$Recall = 1.000 (100\%)$$

Nilai recall 100% menunjukkan bahwa seluruh aktivitas anomali berhasil terdeteksi pada dataset pengujian. Dalam konteks keamanan sistem, recall memiliki peran penting karena berkaitan langsung dengan kemampuan sistem dalam mendeteksi serangan. Nilai False Negative sebesar 0 menunjukkan bahwa tidak terdapat serangan yang terlewat pada skenario simulasi.

4. F1-Score

F1-score merupakan nilai harmonis antara precision dan recall, yang dihitung dengan rumus:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

$$F1 = 2 \times \frac{1 \times 1}{1 + 1}$$

$$F1 = 2 \times \frac{1}{2}$$

$$F1 = 1.000 (100\%)$$

Nilai F1-score sebesar 100% menunjukkan keseimbangan optimal antara kemampuan deteksi dan ketepatan prediksi pada dataset simulasi.

Hasil evaluasi menunjukkan performa model yang sangat tinggi pada lingkungan simulasi terkontrol. Hal ini dipengaruhi oleh proses labeling yang berbasis threshold tertentu (rule_level dan firedtimes), sehingga pemisahan antara kelas normal dan anomali relatif jelas. Perlu dicatat bahwa hasil ini tidak secara otomatis merepresentasikan performa pada lingkungan produksi dengan pola log yang lebih kompleks dan dinamis. Oleh karena itu, hasil evaluasi ini menunjukkan efektivitas model pada dataset penelitian, bukan klaim performa universal. Dengan adanya metrik kuantitatif ini, sistem dengan Machine Learning memiliki keunggulan dibanding sistem tanpa Machine Learning, karena performanya dapat diukur dan dianalisis secara statistik.

4.8.4 Kesimpulan Perbandingan

Berdasarkan hasil perbandingan yang telah dilakukan, sistem tanpa Machine Learning sepenuhnya bergantung pada mekanisme rule-based yang telah ditentukan pada Wazuh. Sistem tersebut mampu menghasilkan alert ketika rule terpenuhi, namun tidak memiliki mekanisme klasifikasi ulang maupun evaluasi performa

secara statistik.

Sebaliknya, sistem dengan integrasi Machine Learning menambahkan lapisan analisis berbasis data melalui algoritma Random Forest. Alert yang dihasilkan oleh Wazuh diproses kembali menggunakan model klasifikasi sehingga keputusan tidak hanya berdasarkan rule statis, tetapi juga berdasarkan pola historis data log keamanan.

Secara kuantitatif, model Machine Learning menunjukkan nilai accuracy, precision, recall, dan F1-score sebesar 1.000 pada dataset simulasi. Hasil ini menunjukkan bahwa model mampu memisahkan kelas normal dan anomali dengan sangat baik pada lingkungan pengujian terkontrol.

Namun demikian, performa tersebut dipengaruhi oleh proses labeling berbasis threshold yang digunakan dalam penelitian ini. Oleh karena itu, hasil evaluasi merepresentasikan efektivitas model pada dataset penelitian, dan bukan klaim performa pada seluruh kondisi operasional.

Dengan demikian, integrasi Machine Learning tidak menggantikan sistem rule-based Wazuh, tetapi meningkatkan mekanisme analisis menjadi sistem dua lapis (rule-based dan data-driven). Pendekatan ini memberikan keunggulan dalam hal validasi alert, evaluasi statistik, serta potensi pengembangan menuju sistem yang lebih adaptif.

BAB V

PENUTUP

5.1 Kesimpulan

Setelah dilakukan implementasi SIEM berbasis wazuh dengan integrasi machine learning untuk deteksi anomali keamanan pada web server, maka dapat diambil kesimpulan sebagai berikut:

1. Sistem Security Information and Event Management (SIEM) berbasis Wazuh berhasil diimplementasikan untuk melakukan monitoring keamanan pada web server secara terpusat melalui integrasi Wazuh Agent, Wazuh Manager, Elasticsearch, dan Kibana.
2. Integrasi Machine Learning menggunakan algoritma Random Forest berhasil diterapkan sebagai lapisan analisis tambahan pada sistem monitoring keamanan. Model dilatih menggunakan pendekatan supervised learning dengan fitur yang diekstrak dari log keamanan, seperti firedtimes, rule_level, hour, is_night, is_ssh, dan log_length.
3. Hasil evaluasi model pada dataset simulasi menunjukkan nilai accuracy, precision, recall, dan F1-score sebesar 1.000 (100%). Hal ini menunjukkan bahwa model mampu mengklasifikasikan aktivitas normal dan anomali dengan baik pada lingkungan pengujian yang digunakan.
4. Berdasarkan hasil perbandingan, sistem tanpa Machine Learning sepenuhnya bergantung pada rule-based detection, sedangkan sistem dengan Machine Learning menambahkan proses klasifikasi ulang terhadap alert. Integrasi Machine Learning memberikan tambahan evaluasi kuantitatif serta validasi berbasis data terhadap alert yang dihasilkan.

5.2 Saran

Setelah dilakukan pembuatan SIEM berbasis wazuh dengan integrasi machine learning untuk deteksi anomali keamanan pada web server, terdapat beberapa saran untuk pengembangan lebih lanjut mengenai infrastruktur ini, diantaranya adalah:

1. Menggunakan dataset yang lebih kompleks dan bervariasi untuk meningkatkan kemampuan generalisasi model.
2. Mengembangkan metode labeling yang lebih mendekati kondisi nyata agar model tidak hanya bergantung pada threshold tertentu.
3. Membandingkan beberapa algoritma Machine Learning untuk memperoleh performa yang lebih optimal.
4. Mengimplementasikan sistem pada lingkungan produksi untuk menguji performa dalam kondisi operasional yang sesungguhnya..

DAFTAR PUSTAKA

- [1] M. T. Andi Yusuf *et al.*, “Laporan Tahunan Honeynet Project BSSN - IHP,” *Direktorat Deteksi Ancaman Badan Siber dan Sandi Negara*, 2019.
- [2] M. A. Hussein and E. K. Hamza, “Secure Mechanism Applied to Big Data for IIoT by Using Security Event and Information Management System (SIEM),” *Int. J. Intell. Eng. Syst.*, 2022, doi: 10.22266/ijies2022.1231.59.
- [3] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, “Deep learning for anomaly detection in log data: A survey,” *Mach. Learn. with Appl.*, vol. 12, no. March, p. 100470, 2023, doi: 10.1016/j.mlwa.2023.100470.
- [4] A. R. Al-harthy, “EUROPEAN JOURNAL OF EMERGING CYBERSECURITY AND INFORMATION PROTECTION AUGMENTING WAZUH SIEM WITH MACHINE LEARNING FOR ADVANCED CYBER THREAT ANALYTICS,” pp. 57–67, 2024.
- [5] Suci Amaliah, M. Nusrang, and A. Aswi, “Penerapan Metode Random Forest Untuk Klasifikasi Varian Minuman Kopi di Kedai Kopi Konijiwa Bantaeng,” *VARIANSI J. Stat. Its Appl. Teach. Res.*, vol. 4, no. 3, pp. 121–127, 2022, doi: 10.35580/variensiunm31.
- [6] A. R. Muhammad, P. Sukarno, and A. A. Wardana, “Integrated Security Information and Event Management (SIEM) with Intrusion Detection System (IDS) for Live Analysis based on Machine Learning,” *Procedia Comput. Sci.*, vol. 217, no. January, pp. 1406–1415, 2022, doi: 10.1016/j.procs.2022.12.339.
- [7] N. Tendikov *et al.*, “Security Information Event Management data acquisition and analysis methods with machine learning principles,” *Results Eng.*, vol. 22, no. March, p. 102254, 2024, doi: 10.1016/j.rineng.2024.102254.
- [8] S. A. Chamkar, M. Zaydi, Y. Maleh, and N. Gherabi, “Improving Threat

- Detection in Wazuh Using Machine Learning Techniques,” *J. Cybersecurity Priv.*, vol. 5, no. 2, pp. 1–25, 2025, doi: 10.3390/jcp5020034.
- [9] M. Affandi, “Analisa Security Information and Event Management (Siem) Menggunakan Elastic Stack Siem Dan Splunk Skripsi,” pp. 1–134, 2022.
- [10] H. Liu and B. Lang, “Machine learning and deep learning methods for intrusion detection systems: A survey,” 2019. doi: 10.3390/app9204396.
- [11] R. Aditya, Y. Muhyidin, and D. Singasatia, “Implementasi Security Information And Event Management (SIEM) Untuk Monitoring Keamanan Server Menggunakan Wazuh,” *Merkurius J. Ris. Sist. Inf. dan Tek. Inform.*, vol. 2, no. 5, pp. 137–144, 2024.
- [12] P. S. Saputra, P. A. Pratama, and L. P. A. S. Tjahyanti, “Perancangan dan Komparasi Web Server Nginx dengan Web Server Apache serta Pemanfaatan Reverse Proxy Server pada Nginx,” *Komteks*, vol. 2, no. 1, pp. 16–21, 2023, [Online]. Available: <https://ejournal.unipas.ac.id/index.php/Komteks/article/view/1307>
- [13] M. Willberg, “Web Application Security Testing With OWASP Top 10 Framework,” *Turku Univ. Appl. Sci.*, p. 33, 2019, [Online]. Available: https://www.theseus.fi/bitstream/handle/10024/170389/Willberg_Mikael.pdf?sequence=2&isAllowed=y
- [14] M. S. Kingsley, “Amazon Web Services (AWS),” in *Textbooks in Telecommunications Engineering*, 2024. doi: 10.1007/978-3-031-33669-0_6.
- [15] Angga Setiyawan, Aryo Pinandito, and Welly Purnomo, “Pengembangan Sistem Informasi Log Management Server Monitoring Menggunakan ELK (Elastic Search, Logstash dan Kibana) Stack pada Aplikasi Padichain di PT. Bank Rakyat Indonesia,” *J. Pengemb. Teknol. Inf. dan Ilmu Komput.*, vol. 7, no. 5, pp. 2142–2151, 2023, [Online]. Available: <http://j-ptiik.ub.ac.id>

- [16] Wijoyo A, Saputra A, Ristanti S, Sya'ban S, Amalia M, and Febriansyah R, "Pembelajaran Machine Learning," *OKTAL (Jurnal Ilmu Komput. dan Sci.*, vol. 3, no. 2, pp. 375–380, 2024, [Online]. Available: <https://journal.mediapublikasi.id/index.php/oktal/article/view/2305>
- [17] N. M. Surbakti *et al.*, "Penggunaan Bahasa Pemrograman Python dalam Pembelajaran Kalkulus Fungsi Dua Variabel," *Algoritma. J. Mat. Ilmu Pengetah. Alam, Kebumihan dan Angkasa*, vol. 2, no. 3, pp. 98–107, 2024, doi: 10.62383/algoritma.v2i3.67.
- [18] A. D. Mulyanto, "Pemanfaatan Bot Telegram Untuk Media," vol. 12, no. 1, pp. 49–54, 2020.
- [19] M. Rizki Andrian Fitra, A. Afrahman S. Effendi, S. Audy Priscilia, and D. Kiswanto, "Uji Penetrasi Menggunakan Hydra Dan Metasploit Pada Protokol Secure Shell," *JATI (Jurnal Mhs. Tek. Inform.*, vol. 9, no. 1, pp. 1017–1024, 2024, doi: 10.36040/jati.v9i1.12583.
- [20] P. Sandika, Y. Muhyidin, and D. Singasatia, "Analisis Keamanan Jaringan Pada Web Server Menggunakan Metasploit Framework Dengan Metode Information System Security Assesment Framework (Issaf) Pada Smkn Xyz," *J. Ilm. Sains Dan Teknol.*, vol. 2, no. 11, pp. 136–150, 2024.

LAMPIRAN

Lampiran 1 Script Pengambilan Dataset

```
import requests
import pandas as pd
from konfigurasi import ELASTIC_BASE_URL, INDEX_PATTERN,
FILE_RAW, JUMLAH_DATA

def ambil_data_elasticsearch():
    url = f"{ELASTIC_BASE_URL}/{INDEX_PATTERN}/_search"

    query = {
        "size": JUMLAH_DATA,
        "sort": [{"@timestamp": {"order": "desc"}}],
        "query": {"match_all": {}}
    }

    print(f"[INFO] Mengambil data dari: {url}")
    resp = requests.get(url, json=query, timeout=30)

    if resp.status_code != 200:
        print("[ERROR] Gagal ambil data!")
        print(resp.status_code, resp.text[:300])
        return None

    data = resp.json()
    hits = data.get("hits", {}).get("hits", [])
    print(f"[INFO] Total dokumen diterima: {len(hits)}")

    records = []
    for hit in hits:
        src = hit.get("_source", {})

        record = {
            "rule_level": src.get("rule", {}).get("level",
0),
            "rule_id": src.get("rule", {}).get("id", ""),
            "rule_description": src.get("rule",
{}).get("description", ""),
            "firedtimes": src.get("rule",
{}).get("firedtimes", 1),
            "timestamp": src.get("@timestamp", ""),
            "agent_id": src.get("agent", {}).get("id", ""),
            "agent_name": src.get("agent", {}).get("name",
""),
            "agent_ip": src.get("agent", {}).get("ip", ""),
            "full_log": src.get("full_log", "")
```

```

        "location": src.get("location", "")
    }

    records.append(record)

df_raw = pd.DataFrame(records)
return df_raw

if __name__ == "__main__":
    df = ambil_data_elasticsearch()
    if df is None or df.empty:
        print("[INFO] Data kosong, tidak ada yang
disimpan.")
        raise SystemExit(0)

    df.to_csv(FILE_RAW, index=False)
    print(f"☒ Data mentah disimpan: {FILE_RAW}")
    print("[INFO] Contoh 5 baris pertama:")
    print(df.head())
    print("[INFO] Total baris:", df.shape[0])

```

Lampiran 2 Script Preprocessing dan Feature Engineering

```
import pandas as pd
from konfigurasi import FILE_RAW, FILE_CLEAN

def preprocessing():
    df_raw = pd.read_csv(FILE_RAW)

    print(f"[INFO] Membaca dataset mentah: {FILE_RAW}")
    print("[INFO] Total baris:", df_raw.shape[0])

    # Buang baris yang kosong pada kolom penting
    df = df_raw.dropna(subset=["timestamp",
"rule_description", "full_log"]).copy()

    # Konversi timestamp
    df["timestamp"] = pd.to_datetime(df["timestamp"],
errors="coerce", utc=True)
    df = df.dropna(subset=["timestamp"])

    # Fitur jam
    df["hour"] = df["timestamp"].dt.hour

    # Fitur malam (00-05)
    df["is_night"] = df["hour"].apply(lambda x: 1 if 0 <=
int(x) <= 5 else 0)

    # Panjang log
    df["log_length"] = df["full_log"].astype(str).apply(len)

    # Apakah mengandung ssh/sshd
    df["is_ssh"] = df.apply(
        lambda row: 1 if ("ssh" in
str(row["rule_description"]).lower()
or "sshd" in
str(row["full_log"]).lower()) else 0,
        axis=1
    )

    # Pastikan numerik
    df["firedtimes"] = pd.to_numeric(df["firedtimes"],
errors="coerce").fillna(1).astype(int)
    df["rule_level"] = pd.to_numeric(df["rule_level"],
errors="coerce").fillna(0).astype(int)

    # Labeling: firedtimes > 10 atau rule_level >= 7 =
anomali
    df["label"] = df.apply(
        lambda row: 1 if (row["firedtimes"] > 10 or
row["rule_level"] >= 7) else 0,
        axis=1
    )
```

```

# Dataset final untuk ML
final_cols = ["firedtimes", "rule_level", "hour",
"is_night", "is_ssh", "log_length", "label"]
df_clean = df[final_cols].copy()

df_clean.to_csv(FILE_CLEAN, index=False)
print(f"☑ Dataset bersih disimpan: {FILE_CLEAN}")

print("[INFO] Distribusi label:")
print(df_clean["label"].value_counts())

print("[INFO] Contoh dataset clean (5 baris):")
print(df_clean.head())

print("[INFO] Total baris clean:", df_clean.shape[0])

if __name__ == "__main__":
    preprocessing()

```

Lampiran 3 Script Pelatihan Model

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from konfigurasi import FILE_CLEAN, FILE_MODEL

def latih_model():
    df = pd.read_csv(FILE_CLEAN)
    print(f"[INFO] Membaca dataset clean: {FILE_CLEAN}")
    print(f"[INFO] Total baris:", df.shape[0])

    X = df.drop(columns=["label"])
    y = df["label"]

    strat = y if y.nunique() > 1 else None
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42, stratify=strat
    )

    print(f"[INFO] Data training:", X_train.shape[0])
    print(f"[INFO] Data testing :", X_test.shape[0])

    print(f"[INFO] Melatih model Random Forest...")
    model = RandomForestClassifier(
        n_estimators=300,
        random_state=42,
        class_weight="balanced"
    )
    model.fit(X_train, y_train)

    joblib.dump(model, FILE_MODEL)
    print(f"☑ Model berhasil disimpan: {FILE_MODEL}")

if __name__ == "__main__":
    latih_model()
```

Lampiran 4 Script Evaluasi Model

```
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report,
confusion_matrix
from konfigurasi import FILE_CLEAN, FILE_MODEL, FILE_REPORT,
FILE_CONFUSION

def evaluasi_model():
    df = pd.read_csv(FILE_CLEAN)
    print(f"[INFO] Membaca dataset clean: {FILE_CLEAN}")
    print("[INFO] Total baris:", df.shape[0])

    X = df.drop(columns=["label"])
    y = df["label"]

    strat = y if y.nunique() > 1 else None
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42, stratify=strat
    )
    model = joblib.load(FILE_MODEL)
    print(f"[INFO] Model dimuat: {FILE_MODEL}")
    y_pred = model.predict(X_test)
    report = classification_report(y_test, y_pred, digits=4)
    cm = confusion_matrix(y_test, y_pred)
    print("\n=== Classification Report ===")
    print(report)
    print("=== Confusion Matrix ===")
    print(cm)

    # Simpan classification report
    with open(FILE_REPORT, "w", encoding="utf-8") as f:
        f.write("Classification Report\n")
        f.write(report + "\n")

    # Simpan confusion matrix ke CSV
    cm_df = pd.DataFrame(cm, index=["aktual_0", "aktual_1"],
        columns=["prediksi_0", "prediksi_1"])
    cm_df.to_csv(FILE_CONFUSION, index=True)

    print(f"\n☑ Report disimpan: {FILE_REPORT}")
    print(f"☑ Confusion matrix disimpan:
{FILE_CONFUSION}")

if __name__ == "__main__":
    evaluasi_model()
```

Lampiran 5 Script Deteksi Realtime dan Notifikasi Telegram

```
import time
import requests
import pandas as pd
import joblib
from konfigurasi import ELASTIC_BASE_URL, INDEX_PATTERN,
FILE_MODEL

def ambil_alert_terbaru():
    url = f"{ELASTIC_BASE_URL}/{INDEX_PATTERN}/_search"
    query = {
        "size": 1,
        "sort": [{"@timestamp": {"order": "desc"}}],
        "query": {"match_all": {}}
    }

    resp = requests.get(url, json=query, timeout=30)
    if resp.status_code != 200:
        print("[ERROR] Gagal ambil alert terbaru:",
resp.status_code, resp.text[:200])
        return None

    data = resp.json()
    hits = data.get("hits", {}).get("hits", [])
    if not hits:
        return None

    return hits[0].get("_source", {})

def ekstrak_fitur(src):
    rule_level = src.get("rule", {}).get("level", 0)
    firedtimes = src.get("rule", {}).get("firedtimes", 1)
    rule_desc = src.get("rule", {}).get("description", "")
    full_log = src.get("full_log", "")
    ts = src.get("@timestamp", "")

    ts_dt = pd.to_datetime(ts, errors="coerce", utc=True)
    hour = int(ts_dt.hour) if pd.notna(ts_dt) else 0
    is_night = 1 if 0 <= hour <= 5 else 0
    log_length = len(str(full_log))
    is_ssh = 1 if ("ssh" in str(rule_desc).lower() or "sshd"
in str(full_log).lower()) else 0

    fitur = {
        "firedtimes": int(firedtimes) if
str(firedtimes).isdigit() else 1,
        "rule_level": int(rule_level) if
str(rule_level).isdigit() else 0,
        "hour": hour,
        "is_night": is_night,
        "is_ssh": is_ssh,
```

```

        "log_length": log_length
    }

    return fitur, ts, rule_desc

def realtime_loop(interval=20):
    model = joblib.load(FILE_MODEL)
    last_ts = None

    print("[INFO] Realtime detector aktif...")
    print("[INFO] Model:", FILE_MODEL)
    print("[INFO] Interval:", interval, "detik\n")

    while True:
        src = ambil_alert_terbaru()
        if src is None:
            print("[INFO] Belum ada alert terbaru.")
            time.sleep(interval)
            continue

        ts = src.get("@timestamp", "")
        if not ts or ts == last_ts:
            time.sleep(interval)
            continue

        last_ts = ts

        fitur, ts, rule_desc = ekstrak_fitur(src)
        X = pd.DataFrame([fitur])
        pred = int(model.predict(X)[0])

        agent = src.get("agent", {}).get("name", "")
        level = src.get("rule", {}).get("level", 0)
        ft = src.get("rule", {}).get("firedtimes", 1)
        full_log = src.get("full_log", "")

        print("\n=====")
        print("[BUKTI 1] Alert terbaru berhasil diambil dari
Elasticsearch")
        print("Waktu      :", ts)
        print("Agent       :", agent)
        print("Rule Level  :", level, "| Firedtimes:", ft)
        print("Rule Desc   :", rule_desc)

        print("\n[BUKTI 2] Fitur hasil preprocessing (input
ke ML):")
        print(fitur)

        print("\n[BUKTI 3] Hasil prediksi ML (1=anomali,
0=normal):", pred)

```

```
print("\n[BUKTI 4] Potongan full_log:")
print(str(full_log)[:200], "...")
print("=====\n")

time.sleep(interval)

if __name__ == "__main__":
    realtime_loop(interval=20)
```

Lampiran 6 Alert JSON Wazuh Tanpa Machine Learning

```
{
  "@timestamp": [
    "2026-02-24T18:24:26.503Z"
  ],
  "agent.id": [
    "002"
  ],
  "agent.ip": [
    "172.31.20.112"
  ],
  "agent.name": [
    "wazuh-agent-1"
  ],
  "data.srcip": [
    "182.9.193.236"
  ],
  "data.srcuser": [
    "root"
  ],
  "decoder.name": [
    "sshd"
  ],
  "decoder.parent": [
    "sshd"
  ],
  "full_log": [
    "Feb 24 18:24:25 ip-172-31-20-112 sshd[2387]: Failed
password for invalid user root from 182.9.193.236 port 58543
ssh2"
  ],
  "GeoLocation.city_name": [
    "Padang"
  ],
  "GeoLocation.country_name": [
    "Indonesia"
  ],
  "GeoLocation.location": [
    {
      "coordinates": [
        100.4354,
        -0.8958
      ],
      "type": "Point"
    }
  ],
  "GeoLocation.region_name": [
    "West Sumatra"
  ],
  "id": [
```

```

        "1771957466.2679"
    ],
    "input.type": [
        "log"
    ],
    "location": [
        "journald"
    ],
    "manager.name": [
        "ip-172-31-18-10"
    ],
    "predecoder.hostname": [
        "ip-172-31-20-112"
    ],
    "predecoder.program_name": [
        "sshd"
    ],
    "predecoder.timestamp": [
        "Feb 24 18:24:25"
    ],
    "rule.description": [
        "sshd: Attempt to login using a non-existent user"
    ],
    "rule.firedtimes": [
        3
    ],
    "rule.gdpr": [
        "IV_35.7.d",
        "IV_32.2"
    ],
    "rule.gpg13": [
        "7.1"
    ],
    "rule.groups": [
        "syslog",
        "sshd",
        "authentication_failed",
        "invalid_login"
    ],
    "rule.hipaa": [
        "164.312.b"
    ],
    "rule.id": [
        "5710"
    ],
    "rule.level": [
        5
    ],
    "rule.mail": [
        false
    ],
    ],

```

```

"rule.mitre.id": [
  "T1110.001",
  "T1021.004"
],
"rule.mitre.tactic": [
  "Credential Access",
  "Lateral Movement"
],
"rule.mitre.technique": [
  "Password Guessing",
  "SSH"
],
"rule.nist_800_53": [
  "AU.14",
  "AC.7",
  "AU.6"
],
"rule.pci_dss": [
  "10.2.4",
  "10.2.5",
  "10.6.1"
],
"rule.tsc": [
  "CC6.1",
  "CC6.8",
  "CC7.2",
  "CC7.3"
],
"timestamp": [
  "2026-02-24T18:24:26.503Z"
],
"_id": "oIv1kJwBX5TxpmKVRgLG",
"_index": "wazuh-alerts-4.x-2026.02.24",
"_score": null
}

```

Lampiran 7 Alert JSON Wazuh dengan Machine Learning

```
{
  "_index": "wazuh-alerts-4.x-2026.02.24",
  "_id": "oIv1kJwBX5TxpmKVRgLG",
  "_version": 1,
  "_source": {
    "predecoder": {
      "hostname": "ip-172-31-20-112",
      "program_name": "sshd",
      "timestamp": "Feb 24 18:24:25"
    },
    "agent": {
      "ip": "172.31.20.112",
      "name": "wazuh-agent-1",
      "id": "002"
    },
    "manager": {
      "name": "ip-172-31-18-10"
    },
    "data": {
      "srcuser": "root",
      "srcip": "182.9.193.236"
    },
    "rule": {
      "mail": false,
      "level": 5,
      "hipaa": [
        "164.312.b"
      ],
      "pci_dss": [
        "10.2.4",
        "10.2.5",
        "10.6.1"
      ],
      "tsc": [
        "CC6.1",
        "CC6.8",
        "CC7.2",
        "CC7.3"
      ],
      "description": "sshd: Attempt to login using a non-existent user",
      "groups": [
        "syslog",
        "sshd",
        "authentication_failed",
        "invalid_login"
      ],
      "nist_800_53": [
        "AU.14",
```

```

        "AC.7",
        "AU.6"
    ],
    "gdpr": [
        "IV_35.7.d",
        "IV_32.2"
    ],
    "firedtimes": 3,
    "mitre": {
        "technique": [
            "Password Guessing",
            "SSH"
        ],
        "id": [
            "T1110.001",
            "T1021.004"
        ],
        "tactic": [
            "Credential Access",
            "Lateral Movement"
        ]
    },
    "id": "5710",
    "gpg13": [
        "7.1"
    ]
},
"decoder": {
    "parent": "sshd",
    "name": "sshd"
},
"full_log": "Feb 24 18:24:25 ip-172-31-20-112
sshd[2387]: Failed password for invalid user root from
182.9.193.236 port 58543 ssh2",
"input": {
    "type": "log"
},
"@timestamp": "2026-02-24T18:24:26.503Z",
"location": "journald",
"id": "1771957466.2679",
"GeoLocation": {
    "city_name": "Padang",
    "country_name": "Indonesia",
    "location": {
        "lon": 100.4354,
        "lat": -0.8958
    },
    "region_name": "West Sumatra"
},
"timestamp": "2026-02-24T18:24:26.503+0000"
},

```

```

"fields": {
  "rule.id": [
    "5710"
  ],
  "rule.gdpr": [
    "IV_35.7.d",
    "IV_32.2"
  ],
  "rule.hipaa": [
    "164.312.b"
  ],
  "GeoLocation.city_name": [
    "Padang"
  ],
  "rule.nist_800_53": [
    "AU.14",
    "AC.7",
    "AU.6"
  ],
  "full_log": [
    "Feb 24 18:24:25 ip-172-31-20-112 sshd[2387]: Failed
password for invalid user root from 182.9.193.236 port 58543
ssh2"
  ],
  "rule.mitre.id": [
    "T1110.001",
    "T1021.004"
  ],
  "manager.name": [
    "ip-172-31-18-10"
  ],
  "predecoder.timestamp": [
    "Feb 24 18:24:25"
  ],
  "rule.pci_dss": [
    "10.2.4",
    "10.2.5",
    "10.6.1"
  ],
  "agent.name": [
    "wazuh-agent-1"
  ],
  "id": [
    "1771957466.2679"
  ],
  "timestamp": [
    "2026-02-24T18:24:26.503Z"
  ],
  "agent.ip": [
    "172.31.20.112"
  ],
}

```

```

"rule.gpg13": [
    "7.1"
],
"GeoLocation.location": [
    {
        "coordinates": [
            100.4354,
            -0.8958
        ],
        "type": "Point"
    }
],
"rule.mitre.technique": [
    "Password Guessing",
    "SSH"
],
"rule.tsc": [
    "CC6.1",
    "CC6.8",
    "CC7.2",
    "CC7.3"
],
"decoder.parent": [
    "sshd"
],
"predecoder.program_name": [
    "sshd"
],
"predecoder.hostname": [
    "ip-172-31-20-112"
],
"rule.mitre.tactic": [
    "Credential Access",
    "Lateral Movement"
],
"input.type": [
    "log"
],
"rule.description": [
    "sshd: Attempt to login using a non-existent user"
],
"data.srcip": [
    "182.9.193.236"
],
"rule.mail": [
    false
],
"GeoLocation.country_name": [
    "Indonesia"
],
"@timestamp": [

```

```

        "2026-02-24T18:24:26.503Z"
    ],
    "agent.id": [
        "002"
    ],
    "decoder.name": [
        "sshd"
    ],
    "location": [
        "journald"
    ],
    "rule.firedtimes": [
        3
    ],
    "rule.groups": [
        "syslog",
        "sshd",
        "authentication_failed",
        "invalid_login"
    ],
    "data.srcuser": [
        "root"
    ],
    "rule.level": [
        5
    ],
    "GeoLocation.region_name": [
        "West Sumatra"
    ]
}
}

```